



BUKU PANDUAN PENGGUNA



PENGEMBANGAN *TRAINING KIT FLIGHT CONTROLLER* *ROBOT FLYING-WINGS* UAV SEBAGAI MEDIA PEMBELAJARAN MATA KULIAH ROBOTIKA

Disusun Oleh:

Zulhakim Seftiyana Roviyan
NIM. 19502241009

Dosen Pembimbing:

Dr. phill. Ir. Mashoedah. S.Pd., M.T.
NIP. 197011082002121003



KATA PENGANTAR

Puji syukur penulis panjatkan kehadirat Allah Swt. yang senantiasa melimpahkan rahmat dan hidayah-Nya sehingga penyusunan buku panduan yang berjudul “Buku Panduan Pengguna Pengembangan *Training Kit Flight Controller Robot Flying Wings UAV* Sebagai Media Pembelajaran Mata Kuliah Robotika” dapat terselesaikan dengan baik. Buku ini disusun dengan tujuan untuk memberikan pemahaman bagi pengguna dalam mengenali dan mengoperasikan *Training Kit Flight Controller Robot Flying Wings UAV*. Melalui buku panduan ini, diharapkan pembaca dapat memperoleh pemahaman yang mendalam tentang konsep dasar, pengoperasian, dan pengembangan teknologi khususnya di bidang *aerial robotic*.

Pada kesempatan ini, penulis ingin mengucapkan terima kasih yang setulus-tulusnya kepada semua pihak yang telah berkontribusi dalam penyusunan buku panduan ini. Penulis sangat menyadari bahwa buku panduan ini tidak akan sempurna tanpa adanya kritik, saran, dan masukan konstruktif dari pembaca. Oleh karena itu, penulis mengharapkan partisipasi aktif dari para pembaca untuk turut berperan dalam meningkatkan kualitas buku panduan ini di masa mendatang. Penulis juga berharap semoga buku panduan ini dapat bermanfaat bagi pembaca, khususnya mahasiswa, dosen, dan praktisi yang tertarik dalam bidang *aerial robotic*. Semoga pengetahuan yang dihadirkan dalam buku ini dapat memberikan kontribusi positif dalam perkembangan teknologi industri khususnya di bidang *aerial robotic*.

Yogyakarta, 25 Juli 2023

Penulis,

Zulhakim Seftiyana Roviyan
NIM. 19502241009



DAFTAR ISI

	Halaman
KATA PENGANTAR.....	2
DAFTAR ISI.....	3
DAFTAR GAMBAR	4
DAFTAR TABEL	5
BAB I TRAINING KIT FLIGHT CONTROLLER ROBOT FLYING WINGS UAV.....	6
A. Pendahuluan.....	6
B. Pengertian <i>Training Kit Flight Controller</i>	7
C. Pengertian Mata Kuliah Robotika.....	8
D. Prinsip Kerja <i>Training Kit Flight Controller</i>	9
BAB II ELEMEN-ELEMEN TRAINING KIT FLIGHT CONTROLLER ROBOT FLYING WINGS UAV	13
A. Bagian-Bagian <i>Training Kit Flight Controller</i>	13
B. Skematik Elektronik <i>Training Kit Flight Controller</i>	21
C. Desain 3D Robot <i>Flying Wings</i>	22
D. Sistem Kontrol LQR (<i>Linear Quadratic Regulation</i>)	24
BAB III PENGGUNAAN DAN PENGOPERASIAN TRAINING KIT FLIGHT CONTROLLER ROBOT FLYING WINGS UAV.....	28
A. Wiring Diagram <i>Training Kit Flight Controller</i>	28
B. Konfigurasi Elektronik <i>Training Kit Flight Controller</i>	29
C. Pemrograman <i>Training Kit Flight Controller</i>	43
D. Pengujian <i>Training Kit Flight Controller</i>	64
E. Aplikasi Robot <i>Flying Wings UAV</i>	74
PENUTUP.....	75
DAFTAR PUSTAKA.....	76

DAFTAR GAMBAR

Gambar 1. Arsitektur Elektronis <i>Flight Controller</i>	9
Gambar 2. Alir Algoritma Pemrograman <i>Flight Controller</i>	10
Gambar 3. Bagian <i>Training Kit Flight Controller Robot Flying Wings</i>	13
Gambar 4. Bagian-Bagian Remote Kendali <i>Training Kit Flight Controller Robot Flying Wings</i>	14
Gambar 5. Bagian-Bagian Blok Elektronik <i>Training Kit Flight Controller Robot Flying Wings</i>	16
Gambar 6. Rangkaian Skematik <i>Training Kit Flight Controller Robot Flying Wings UAV</i>	21
Gambar 7. Desain 3D Pesawat <i>Flying Wings</i>	23
Gambar 8. Dimensi Pesawat <i>Flying Wings</i>	23
Gambar 9. <i>State Space Autonomous Flying Wing</i>	24
Gambar 10. Diagram Blok Sistem Kendali <i>Flying Wings</i>	26
Gambar 11. Wiring Diagram <i>Flight Controller Robot Flying Wings</i>	28
Gambar 12. Kode Program Perhitungan Nilai K di Matlab	67
Gambar 13. Kode Program Simulasi Matlab Kendali LQR	68
Gambar 14. Hasil Simulasi Matlab untuk Nilai K	69
Gambar 15. Grafik Karakteristik Respon Sistem Kendali LQR	69
Gambar 16. Potongan Kode Program Pengendalian Gerak Rotasi.....	71
Gambar 17. Posisi CH5 dan CH6 pada Remote Kendali.....	72

DAFTAR TABEL

Tabel 1. Silabus Mata Kuliah Robotika 2019	9
Tabel 2. Persamaan Momen Inersia	65
Tabel 3. Komponen Inersia <i>Training Kit</i> Pesawat <i>Flying-Wings</i>	66
Tabel 4. Karakteristik Respon Sistem Kendali LQR	70
Tabel 4. Tabel 4. Karakteristik Respon Sistem Kendali LQR.....	62
Tabel 4. Gerakan Elevon (Mode Manual).....	72
Tabel 5. Gerakan Elevon (Mode <i>Stabilize</i>)	73

BAB I

TRAINING KIT FLIGHT CONTROLLER ROBOT FLYING WINGS UAV

A. Pendahuluan

Proses pembelajaran di perguruan tinggi dituntut untuk terus berkembang mengikuti pesatnya perkembangan teknologi terkini. Pengoptimalan proses pembelajaran dapat dilakukan dengan pemanfaatan media pembelajaran sebagai alat bantu dalam menyampaikan materi pembelajaran. Media pembelajaran merupakan bagian penting dari proses pembelajaran karena dapat membantu memperjelas makna dan menyajikan informasi dalam bentuk yang menarik (Tafonao, 2018), sehingga media pembelajaran diharuskan untuk terus beradaptasi sesuai dengan kondisi perkembangan teknologi terkini. Teknologi Pesawat Tanpa Awak atau *Unmanned Aerial Vehicle* (UAV) menjadi salah satu teknologi yang populer dan banyak digunakan di berbagai bidang seperti pertanian, pemetaan, pertahanan, dan transportasi (Ardiantara et al., 2014).

Training kit flight controller merupakan salah satu media pembelajaran yang dapat membantu optimalisasi pemahaman konsep *flight controller* secara lebih efektif dan efisien sesuai dengan perkembangan teknologi terkini. *Training kit flight controller* terdiri dari mikrokontroler, sensor-sensor, motor brushless, motor servo, dan remote control sebagai pengendali jarak jauh yang digunakan oleh operator atau pilot. Melalui *training kit flight controller*, mahasiswa dapat melakukan praktikum yang diperlukan untuk memahami cara kerja *flight controller* dan menganalisis bagaimana *flight controller* mengolah data dari sensor-sensor untuk mengendalikan Pesawat Tanpa Awak baik secara manual maupun otomatis. Media pembelajaran *flight controller* yang dikembangkan diharapkan mampu memaksimalkan capaian kompetensi mahasiswa pada bidang robotika khususnya mata kuliah robotika bagian materi *aerial robotic*. Sebagai sebuah investasi pendidikan yang berharga, *Training Kit Flight Controller Robot Flying Wings UAV* diharapkan dapat memperluas wawasan dan keterampilan mahasiswa di era teknologi yang terus berkembang pesat.

B. Pengertian *Training Kit Flight Controller*

Training kit flight controller robot flying wings UAV merupakan sebuah media pembelajaran yang inovatif dan dirancang untuk memberikan pengalaman belajar secara interaktif serta aplikatif. *Training kit flight controller* ini juga merupakan solusi yang efektif bagi institusi pendidikan dalam menyediakan sarana praktik yang aman untuk mahasiswa. *training kit flight controller* dirancang dengan menggunakan sensor IMU (*Inertial Measurement Unit*) sebagai masukan untuk menyediakan data *yaw*, *pitch*, dan *roll* dari sebuah pesawat. Mikrokontroler Teensy 3.5 digunakan sebagai unit pemrosesan data dengan prosesor ARM Cortex-M4 dan *unit floating point* yang berkualitas tinggi, sehingga cocok untuk aplikasi pengembangan seperti *flight controller*. Selain itu, sistem kontrol *Linear Quadratic Regulator* digunakan pada *Training Kit Flight Controller* dengan algoritma matematika *optimal control* untuk menentukan pola kontrol yang dapat mencapai kestabilan yang baik. Aktuator berupa motor servo digunakan sebagai komponen untuk mengendalikan gerak *elevon* pesawat yang didasarkan data yang didapatkan dari sensor masukan *accelerometer* dan *gyroscope*. Motor *brushless direct current* (BLDC) juga digunakan sebagai sistem propulsi untuk menggerakkan propeller dalam memberikan daya dorong (*thrust*) pada pesawat. *Training Kit Flight Controller* juga dilengkapi dengan radio control sebagai sebuah yang digunakan untuk mengendalikan perangkat elektronik dari jarak jauh melalui gelombang radio. Radio control bekerja dengan menggunakan perangkat pengirim sinyal (*transmitter*) dan perangkat penerima sinyal (*receiver*). Transmitter berfungsi untuk mengirimkan sinyal gelombang rasio yang mengandung informasi kontrol, sedangkan receiver berfungsi untuk menerima sinyal tersebut dan mengubah ke dalam bentuk sinyal *Pulse Width Modulation* yang dikirimkan ke mikrokontroler Teensy 3.5 untuk diproses dalam mengendalikan pesawat.

Selain sebagai media pembelajaran, *training kit* ini dapat menjadi wahana eksplorasi dan inspirasi bagi para calon ilmuwan muda. Pengalaman berinteraksi langsung dengan teknologi *aerial robotic* dapat memicu minat dan semangat dalam mengejar karir di bidang ilmu pengetahuan dan teknologi, terutama dalam pengembangan pemrograman *aerial robotic* khususnya pesawat

model *flying wings* yang memiliki potensi besar dalam berbagai aplikasi. Mahasiswa dapat memanfaatkan fasilitas yang ada dalam *Training Kit Flight Controller* untuk mengembangkan algoritma dan melakukan beberapa eksperimen untuk kemajuan teknologi *aerial robotic* secara keseluruhan. Melalui *Training Kit Flight Controller*, diharapkan muncul generasi-generasi baru inovator dan pemimpin di bidang teknologi yang mampu menghadapi tantangan perkembangan era industri 4.0 dengan solusi yang kreatif.

C. Pengertian Mata Kuliah Robotika

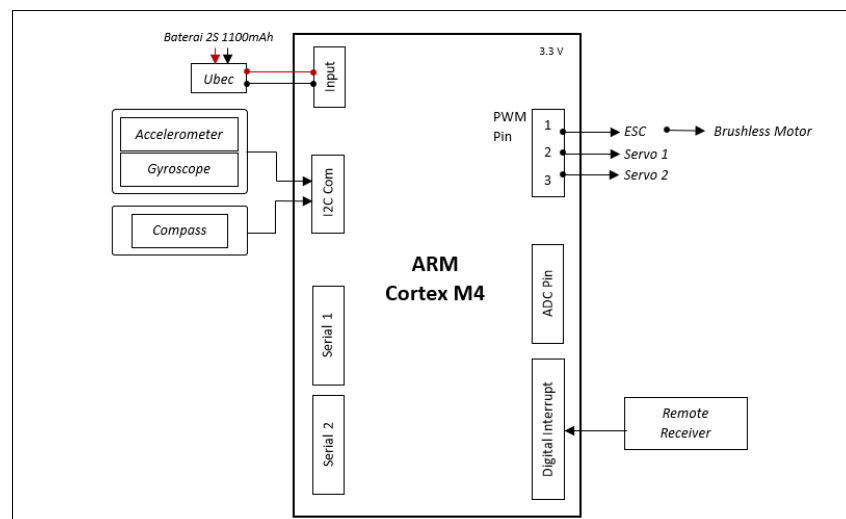
Mata Kuliah Robotika merupakan mata kuliah pilihan dengan bobot 2 SKS praktek yang menyajikan pemahaman dasar-dasar robotika, pemodelan, dan lintasan kinematik atau dinamis. Mata Kuliah Robotika mempelajari sistem gerak, sensor, aktuator, kontrol robot, mobile robot, robot manipulator (*arm*), dan robot terbang (*x-copter*). *Training Kit Flight Controller* pada robot *flying-wings* UAV dikembangkan dengan mengacu pada Silabus Mata Kuliah Robotika Program Studi Pendidikan Teknik Elektronika Fakultas Teknik Universitas Negeri Yogyakarta. Rincian sub kompetensi dan indikator pencapaian mata kuliah robotika dapat dilihat pada tabel 1 berikut.

Tabel 1. Silabus Mata Kuliah Robotika Tahun 2019

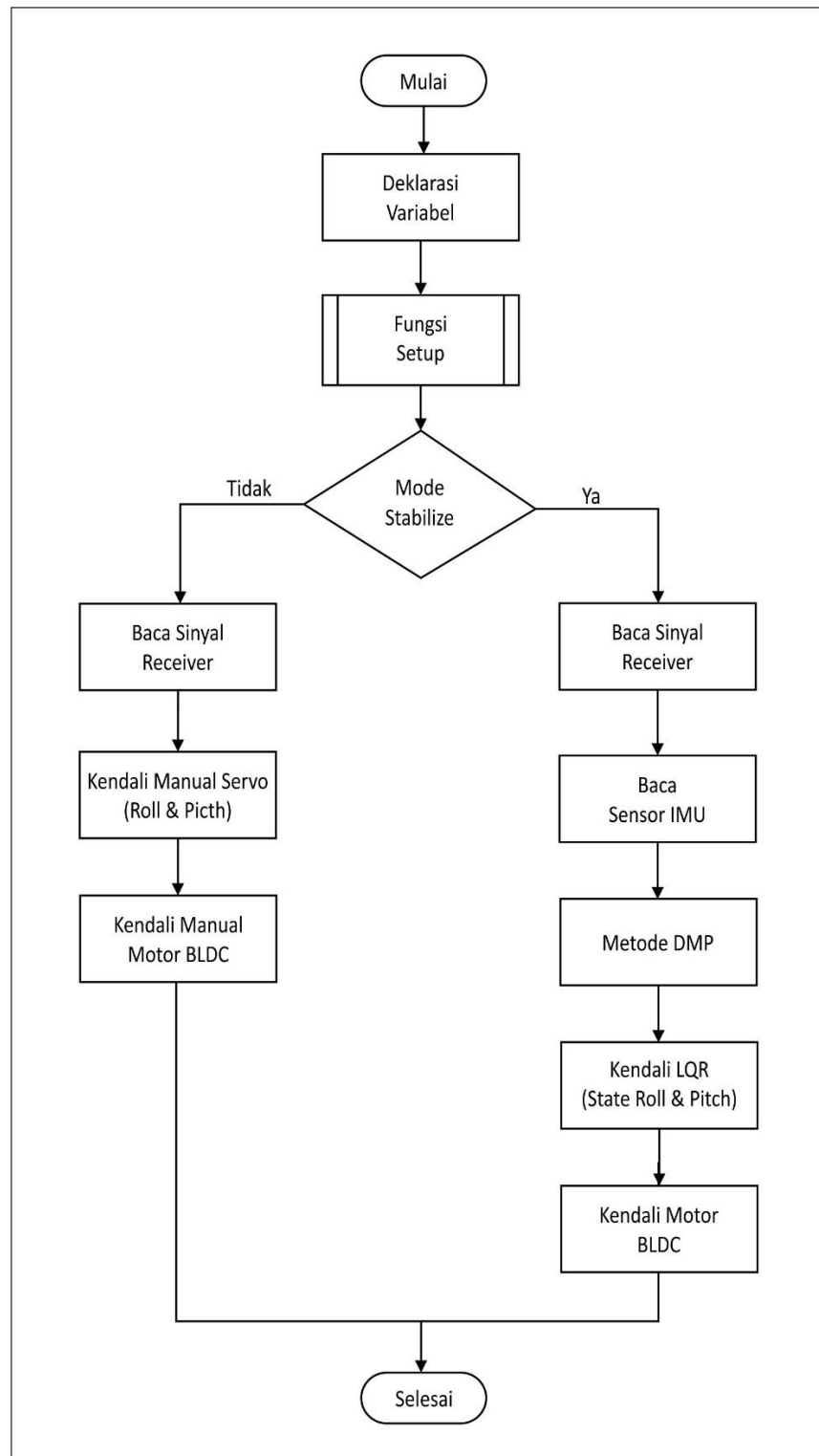
No.	Sub Kompetensi	Indikator Pencapaian
1	Konsep Dasar Robotika	Mahasiswa mampu menjelaskan konsep robotika
2	Aktuator dan Kontroler	Mahasiswa mampu membuat model robot
3	Kinematika dan Dinamika Robot	Mahasiswa mampu menyelesaikan persamaan kinematika dan persamaan dinamis
4	Teknik Pemrograman Robot	Mahasiswa mampu menentukan sensor dan tranduser sebagai kebutuhan sistem robotika
5	Robot Beroda Robot Berkaki Robot Terbang	Mahasiswa mampu merancang sistem robotika

D. Prinsip Kerja *Training Kit Flight Controller*

Prinsip kerja *Training Kit Flight Controller robot flying wings UAV* didasarkan pada integrasi dan interaksi antara beberapa komponen utama yang bekerja secara sinergis untuk mencapai tujuan utama, yaitu mengendalikan dan mengoperasikan pesawat *flying wings* secara stabil. Prinsip kerja *training kit flight controller* diilustrasikan dalam gambar arsitektur elektronis dan alir algoritma pemrograman berikut.



Gambar 1. Arsitektur Elektronis *Flight Controller*



Gambar 2. Alir Algoritma Pemrograman *Flight Controller*

Secara keseluruhan, prinsip kerja *training kit flight controller* melibatkan interaksi yang kompleks dan terkoordinasi antara masukan, proses, dan keluaran. Integrasi ini memungkinkan pesawat *flying wings* untuk dioperasikan melalui pengendalian menggunakan perangkat radio control dengan kestabilan dan kinerja yang baik di berbagai kondisi. Proses kerja *training kit flight controller* dijelaskan pada beberapa tahapan berikut.

1. Masukan (*input*)

Masukan utama dari *training kit flight controller* adalah sensor *Inertial Measurement Unit* (IMU) yang berperan penting dalam menyediakan data *yaw*, *pitch*, dan *roll*. Data tersebut menggambarkan orientasi dan pergerakan pesawat dalam tiga sumbu yang berbeda yakni x, y, dan z. Sensor *accelerometer* dan *gyroscope* mendeteksi perubahan kecepatan linier dan kecepatan sudut pesawat, yang kemudian diolah menjadi informasi mengenai posisi dan arah pesawat.

Training kit flight controller juga memiliki masukan lain berupa sinyal radio control yang memungkinkan pengendalian pesawat dari jarak jauh melalui gelombang radio. Pengguna dapat mengontrol pesawat menggunakan perangkat pengirim sinyal (*transmitter*) yang mengirimkan sinyal gelombang radio dengan informasi kontrol, dimana sinyal tersebut akan diterima oleh perangkat penerima sinyal (*receiver*) yang kemudian mengubahnya menjadi sinyal *Pulse Width Modulation* (PWM). Sinyal PWM tersebut yang diproses oleh mikrokontroler Teensy 3.5 dalam mengendalikan pesawat sesuai dengan instruksi dari *transmitter*.

2. Proses (*processing*)

Mikrokontroler Teensy 3.5 merupakan unit yang bertanggung jawab untuk menerima dan memproses data yang didapatkan dari inputan. Mikrokontroler tersebut menjadi otak sistem yang menginterpretasikan data sensor dan sinyal radio control untuk pengambilan keputusan mengenai kontrol pesawat berdasarkan algoritma yang telah ditanamkan. Algoritma kontrol *Linear Quadratic Regulator* (LQR) merupakan salah satu algoritma yang mempunyai peranan penting dalam tahapan proses untuk menentukan pola kontrol pesawat dalam mencapai kestabilan yang

baik. Algoritma ini menggunakan konsep matematika optimal control untuk menghitung nilai input yang tepat berdasarkan state dan goal system yang diinginkan.

3. Keluaran (*output*)

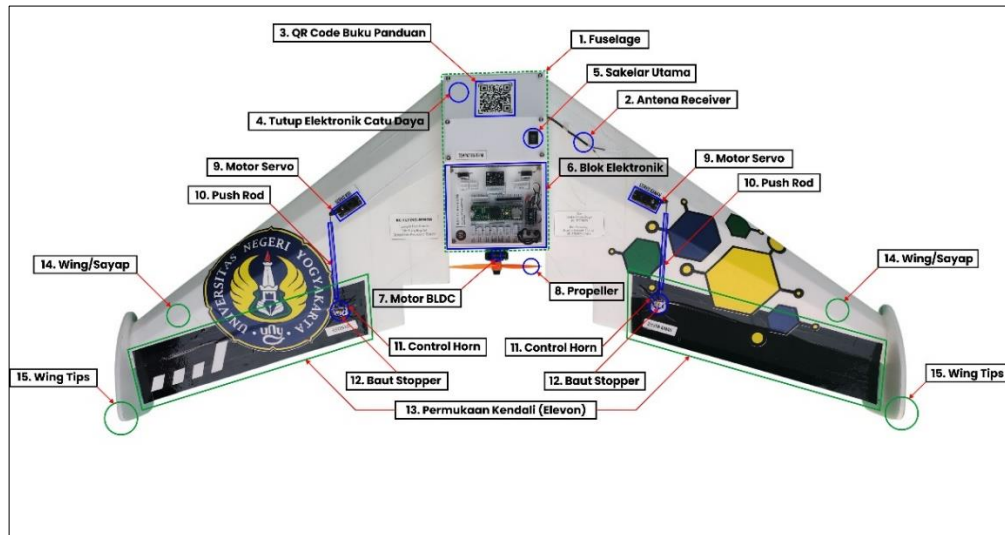
Keluaran dari *training kit flight controller* berupa dua buah motor servo dan satu motor *Brushless Direct Current* (BLDC). Motor servo digunakan sebagai aktuator untuk mengendalikan gerak elevon berdasarkan input dari sensor dan hasil perhitungan oleh *processor* atau mikrokontroler. Elevon sendiri merupakan bagian dari pesawat yang berfungsi untuk mengatur kemiringan dan gerakan pesawat pada sumbu *pitch* dan *roll*. Sedangkan, motor BLDC disini berperan sebagai sistem propulsi yang menggerakkan propeller pesawat dalam memberikan daya dorong (*thrust*). Pengaturan putaran motor BLDC dilakukan oleh komponen *Electronic Speed Controller* (ESC) berdasarkan pada inputan sinyal dari radio control yang diproses oleh mikrokontroler.

BAB II

ELEMEN-ELEMEN *TRAINING KIT FLIGHT CONTROLLER ROBOT FLYING WINGS UAV*

A. Bagian-Bagian *Training Kit Flight Controller*

Bagian-bagian dari *Training Kit Flight Controller Robot Flying Wings UAV* di ilustrasikan pada gambar berikut.



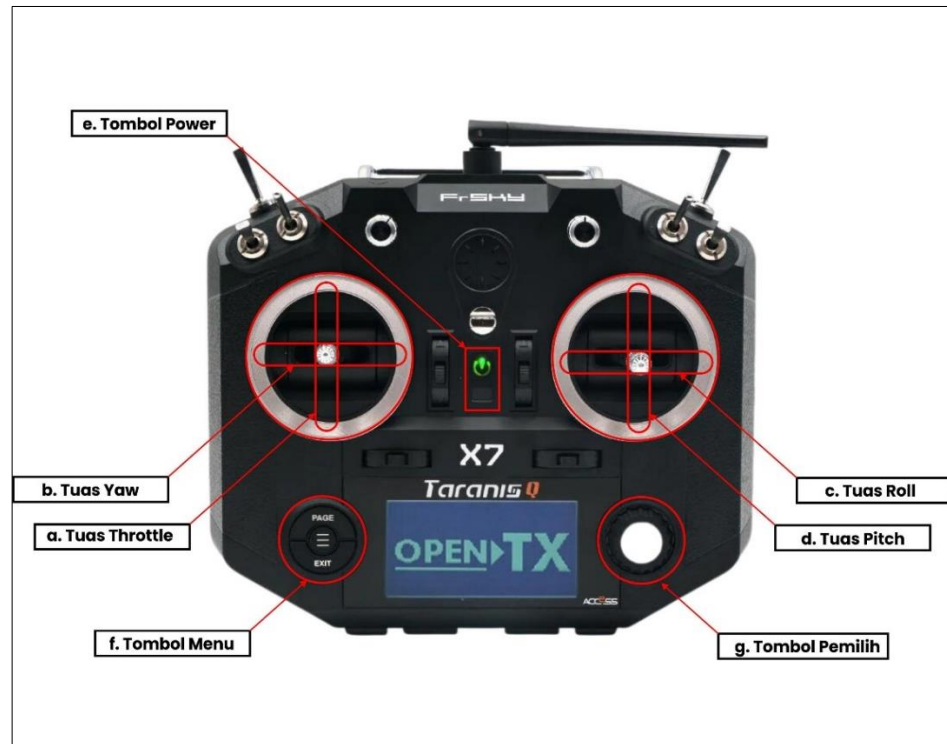
Gambar 3. Bagian-Bagian *Training Kit Flight Controller Robot Flying Wings*

1. *Fuselage*

Fuselage adalah “badan” utama pesawat yang menjadi fondasi sistem avionik (elektronik penerbangan), serta berperan dalam menyediakan ruang untuk pemasangan sayap (*wing*).

2. *Antena Receiver*

Antena receiver merupakan bagian dari perangkat penerima sinyal yang berfungsi untuk menangkap dan mengumpulkan sinyal yang dikirimkan oleh *transmitter*. Sinyal tersebut kemudian akan diteruskan ke perangkat penerima (*receiver*) yang lebih kompleks untuk diproses menjadi sinyal keluaran yang berbeda. Kualitas *antena receiver* yang baik dapat meningkatkan sensitivitas yang lebih baik dalam menjaga koneksi yang stabil. Bagian dari *transmitter* yang digunakan pada *training kit flight controller* ditunjukkan pada gambar berikut.



Gambar 4. Bagian-Bagian Remote Kendali *Training Kit Flight Controller*
Robot *Flying Wings*

a. Tuas *Throttle*

Tuas *throttle* merupakan bagian dari remote control (*transmitter*) yang digunakan untuk mengatur daya dorong atau dalam hal ini adalah kecepatan motor *brushless direct current*.

b. Tuas *Yaw*

Tuas *yaw* merupakan bagian dari remote control (*transmitter*) yang digunakan untuk mengendalikan gerakan miring (*yaw*) pada pesawat *flying wing*. Gerakan *yaw* sendiri adalah gerakan mengelilingi sumbu vertikal, yang membuat pesawat berputar atau miring ke kiri atau kanan tanpa mengubah ketinggiannya.

c. Tuas *Roll*

Tuas *roll* merupakan bagian dari remote control (*transmitter*) yang digunakan untuk mengendalikan gerakan *roll* atau berputar pada pesawat *flying wing*. Gerakan *roll* merupakan gerakan mengelilingi sumbu longitudinal untuk mengubah kemiringan atau posisi sayap, sehingga pesawat akan berputar di sekitar sumbu utama pesawat.

d. Tuas *Pitch*

Tuas *pitch* merupakan bagian dari remote control (*transmitter*) yang digunakan untuk mengendalikan gerakan *pitch* atau naik-turun pada pesawat *flying wing*. Gerakan *pitch* adalah gerakan mengelilingi sumbu lateral pesawat, yang membuat pesawat naik atau turun dengan mengubah posisi hidung (*nose*) pesawat.

e. Tombol Power

Tombol power digunakan untuk menghidupkan atau mematikan remote control (*transmitter*).

f. Tombol Menu

Tombol menu remote control (*transmitter*) digunakan untuk mengakses berbagai menu dan pengaturan terkait konfigurasi yang dibutuhkan.

g. Tombol Pemilih

Tombol pemilih digunakan pada remote control (*transmitter*) untuk memilih opsi pada saat melakukan konfigurasi.

3. QR Code Buku Panduan

QR code pada *training kit flight controller* merupakan jenis kode dua dimensi yang berfungsi untuk menyimpan informasi terkait buku panduan pengguna *training kit flight controller*.

4. Tutup Elektronik Catu Daya

Tutup yang terletak pada *fuselage* ini berfungsi untuk melindungi komponen catu daya (baterai dan *step-down*) dari risiko lingkungan eksternal.

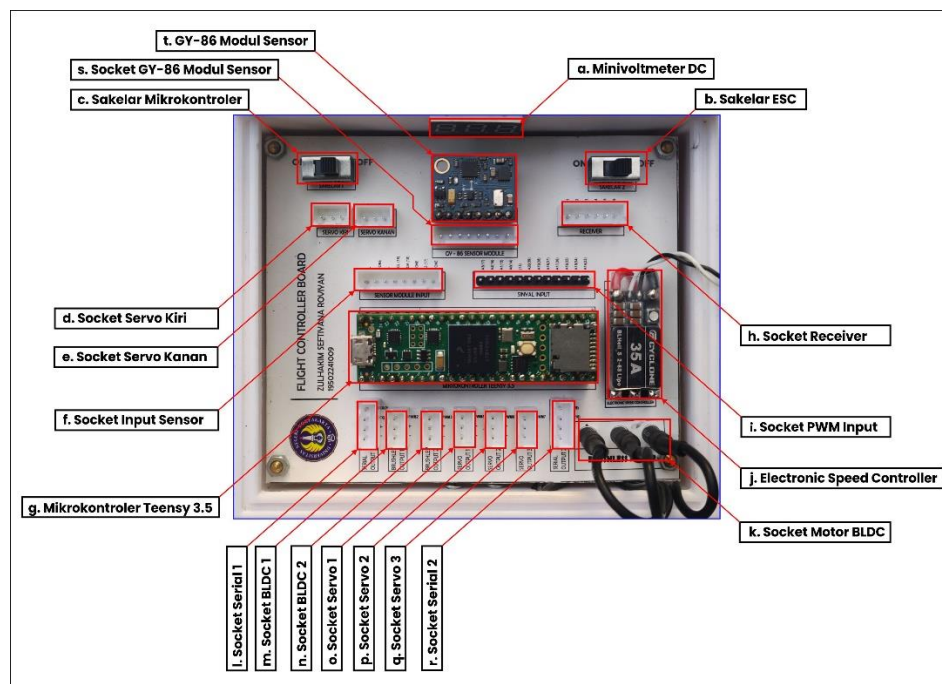
5. Saklar Utama

Saklar utama pada *training kit flight controller* berfungsi sebagai pengontrol aliran listrik utama pada sistem elektronik, dimana saklar ini memutus dan menghubungkan arus dari baterai sebelum mengalir ke komponen yang digunakan pada *training kit flight controller*.

6. Blok Elektronik

Papan komponen elektronik berfungsi sebagai platform fisik untuk pemasangan dan koneksi berbagai komponen elektronik dalam *training kit*

flight controller. Berikut ini adalah gambar komponen yang terpasang pada papan komponen elektronik *training kit flight controller*.



Gambar 5. Bagian-Bagian Blok Elektronik *Training Kit Flight Controller* Robot *Flying Wings*

a. Minivoltmeter DC

Minivoltmeter ini digunakan untuk mengukur tegangan listrik catu daya baterai pada *training kit flight controller*. Penggunaan minivoltmeter tersebut dalam rangka untuk memantau kapasitas catu daya baterai yang digunakan pada *training kit flight controller*, agar performa dan kualitas baterai tetap pada kondisi yang maksimal.

b. Saklar ESC

Saklar ini digunakan untuk memutus dan menghubungkan arus dari baterai sebelum mengalir ke komponen *electronic speed controller* pada *training kit flight controller*.

c. Saklar Mikrokontroler

Saklar ini digunakan untuk memutus dan menghubungkan arus dari *step-down* sebelum mengalir ke komponen *electronic speed controller* pada *training kit flight controller*.

d. *Socket Servo Kiri*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan servo kiri pada *training kit flight controller*. Penggunaan konektor pada *training kit flight controller* difungsikan untuk melatih psikomotorik mahasiswa saat melakukan praktikum karena terdapat kegiatan lepas-pasang yang melibatkan aspek kognitif (pikiran) dan motorik (gerakan).

e. *Socket Servo Kanan*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan servo kanan pada *training kit flight controller*.

f. *Socket Input Sensor*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan sensor GY-86 module pada *training kit flight controller*.

g. *Mikrokontroler Teensy 3.5*

Mikrokontroler teensy 3.5 merupakan sebuah board pengembangan yang didesain oleh Paul Stoffregen dan PJRC. Teensy 3.5 termasuk salah satu mikrokontroler yang multifungsi dan dapat digunakan untuk berbagai proyek seperti *Robotics*, *Internet of Things*, *Computer Vision*, dan *Unmanned Aerial Vehicle*. Teensy 3.5 memiliki prosesor ARM Cortex-M4 dengan *unit floating point* yang berkualitas tinggi, sehingga cocok untuk aplikasi pengembangan seperti *flight controller*.

h. *Socket Receiver*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan *receiver* pada *training kit flight controller*.

i. *Socket PWM Input*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan pin PWM input mikrokontroler Teensy 3.5 pada *training kit flight controller*.

j. *Electronic Speed Controller*

Electronic Speed Controller (ESC) merupakan sebuah modul elektronik yang bertanggung jawab untuk mengontrol keluaran daya

dan kecepatan rotasi motor sebagai respon terhadap perintah throttle. Electronic Speed Controller memiliki konsep mengubah arus searah (DC) menjadi arus (AC) 3-phase, dengan komponen utama berupa Mikrokontroler dan *Metal Oxide Semiconductor Field Effect Transistor* (MOSFET).

k. *Socket Motor BLDC*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan motor BLDC pada *training kit flight controller*.

l. *Socket Serial 1*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan pin RX dan TX mikrokontroler Teensy 3.5 pada *training kit flight controller*.

m. *Socket BLDC 1*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan ESC pada *training kit flight controller*, dimana ESC itu sendiri yang berfungsi mengatur kecepatan putar motor BLDC.

n. *Socket BLDC 2*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan ESC pada *training kit flight controller*, dimana ESC itu sendiri yang berfungsi mengatur kecepatan putar motor BLDC.

o. *Socket Servo 1*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan pin PWM output mikrokontroler Teensy 3.5 pada *training kit flight controller* yang digunakan untuk menggerakkan motor servo.

p. *Socket Servo 2*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan pin PWM output mikrokontroler Teensy 3.5 pada *training kit flight controller* yang digunakan untuk menggerakkan motor servo.

q. *Socket Servo 3*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan pin PWM output mikrokontroler Teensy 3.5 pada *training kit flight controller* yang digunakan untuk menggerakkan motor servo.

r. *Socket Serial 2*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan pin RX dan TX mikrokontroler Teensy 3.5 pada *training kit flight controller*.

s. *Socket GY-86 Modul Sensor*

Socket ini digunakan sebagai konektor yang terhubung langsung dengan pin pada GY-86 modul sensor.

t. *GY-86 Modul Sensor*

GY-86 merupakan sebuah modul sensor yang mengkombinasikan tiga sensor yakni MPU6050 (*accelerometer* dan *gyroscope*), HMC5883L (*magnetometer*), dan MS5611 (*barometer*). Modul sensor GY-86 dirancang untuk memberikan solusi yang praktis dalam memperoleh data orientasi, gerakan, dan ketinggian. Modul sensor GY-86 pada *training kit flight controller* digunakan untuk membaca perubahan parameter orientasi pesawat (*roll*, *pitch*, dan *yaw*).

7. **Motor BLDC**

Motor diartikan sebagai komponen elektronik yang berfungsi untuk mengubah energi listrik menjadi energi mekanik melalui proses elektromagnetik (Yasa, 2022). Motor *Brushless Direct Current* (BLDC) merupakan jenis motor listrik yang bekerja menggunakan sistem komutasi secara elektronik dengan sistem perpindahan medan magnet antara stator dan rotor tanpa menggunakan sikat (*brush*).

8. **Propeller**

Propeller merupakan sebuah komponen yang dirancang untuk menghasilkan gaya dorong (*thrust*) dengan memanfaatkan gaya aksi-reaksi. Propeller pada *training kit flight controller* berputar karena digerakan menggunakan motor BLDC.

9. **Motor Servo**

Motor servo merupakan salah satu jenis motor dengan sistem closed feedback, dimana posisi motor akan diinformasikan kembali ke rangkaian kontrol yang ada di dalam motor servo (Salim et al., 2020). Motor servo memiliki kemampuan untuk kontrol yang tepat dari posisi sudut dan

kecepatan, dengan kata lain motor servo dapat berputar pada sudut yang tepat serta mempertahankan posisi tersebut.

10. Pushrod

Pushrod merupakan komponen mekanis yang digunakan sebagai sistem penggerak untuk mentransfer gaya dari motor servo ke *control horn*.

11. Control Horn

Control horn adalah komponen yang berfungsi sebagai pengubah gerakan rotasi linear dari servo menjadi gerakan rotasi permukaan kendali (elevon).

12. Baut Stopper

Baut ini digunakan pada *training kit flight controller* untuk mengatur dan membatasi panjang efektif dari *pushrod* agar kontrol elevon lebih akurat dan pergerakan pesawat responsif.

13. Elevon

Elevon merupakan permukaan kontrol kombinasi elevator dan aileron yang berfungsi untuk mengendalikan pergerakan dan keseimbangan pesawat. Elevon menjadi salah satu bagian dari pesawat yang berperan dalam mengubah sudut atau kemiringan pesawat di sekitar sumbu longitudinal (*roll*) dan sumbu lateral (*pitch*).

14. Wing/Sayap

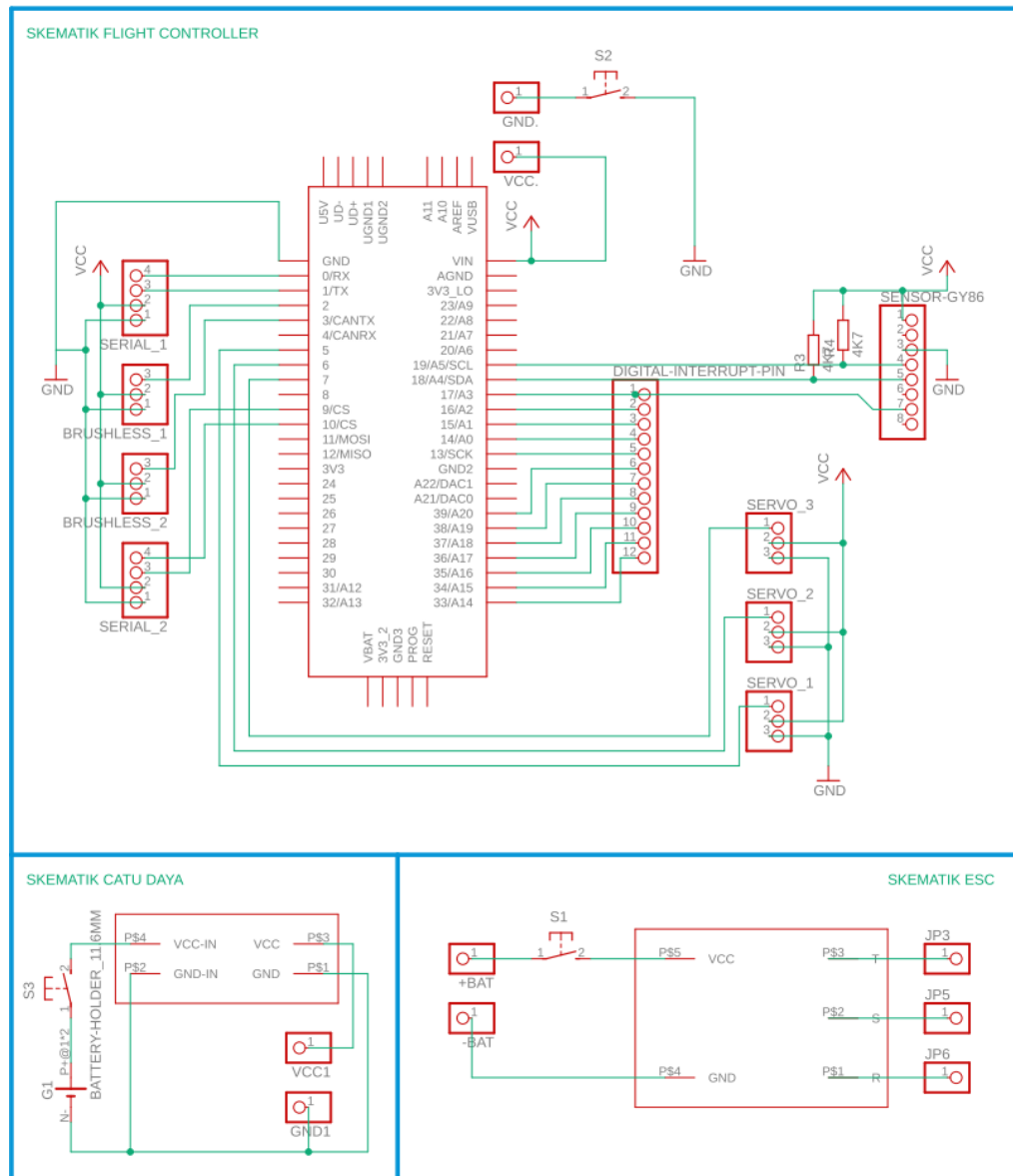
Wing atau sayap merupakan bagian utama dari pesawat untuk menghasilkan gaya angkat (*lift*) yang membuat pesawat dapat terbang. Sayap atau *wing* menjadi elemen sentral dalam desain pesawat, dimana perancangan *wing* yang tepat sangat penting untuk mengoptimalkan kinerja aerodinamis.

15. Wing Tips

Wing tips merupakan bagian ujung pesawat yang memiliki peran penting dalam aerodinamika dan performa penerbangan pesawat, dimana desain *wing tips* dapat mempengaruhi efisiensi, stabilitas, dan karakteristik penerbangan pesawat.

B. Skematik Elektronik Training Kit Flight Controller

Gambar skematik elektronik yang digunakan pada *training kit flight controller* tersusun dengan komponen utama mikrokontroler Teensy 3.5 seperti pada gambar berikut.

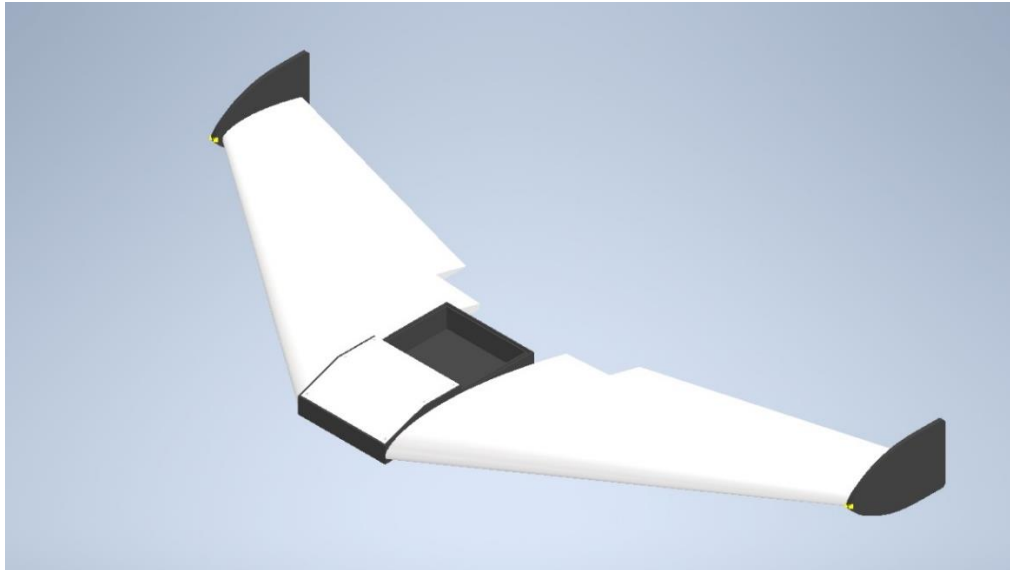


Gambar 6. Rangkaian Skematik *Training Kit Flight Controller Robot Flying Wings UAV*

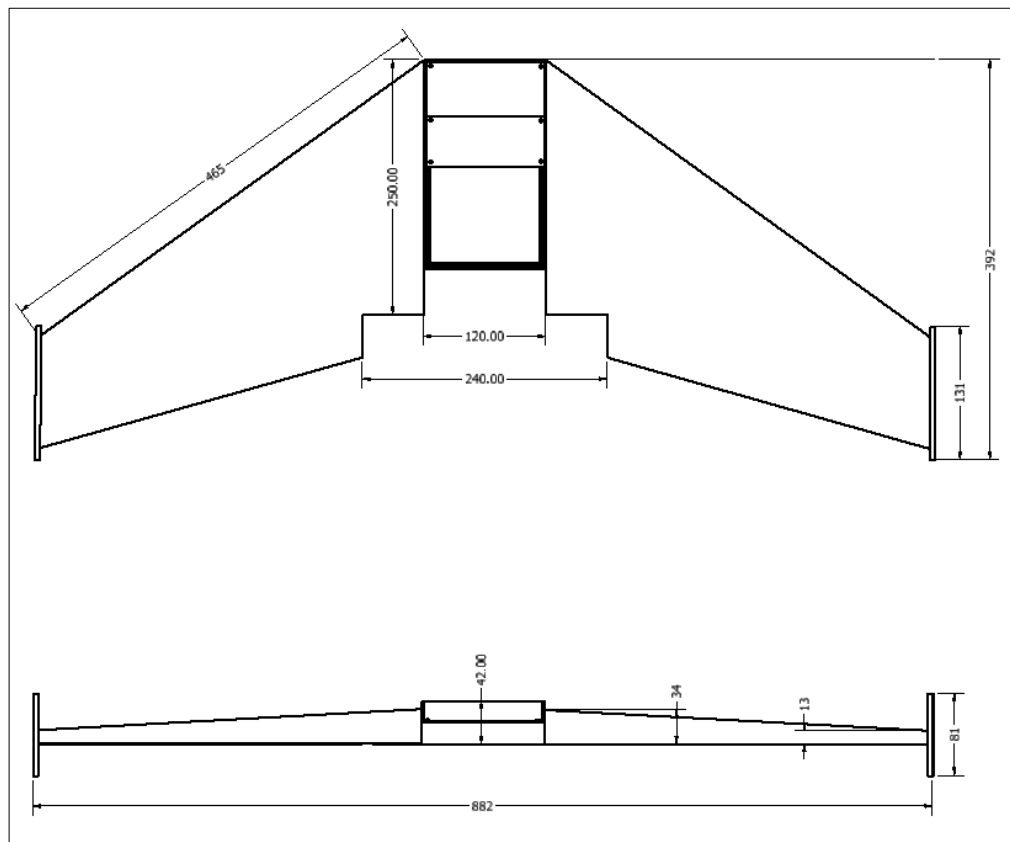
Perangkat keras yang digunakan dalam mengolah algoritma pada *flight controller* ini adalah mikrokontroler ARM Cortex-M4 dengan kecepatan kalkulasi komputasi atau *rated speed* 96 MHz dan dapat ditingkatkan melalui *overclocking* hingga 120 MHz. Selain itu mikrokontroler ini memiliki *flash memory* sebesar 512 KBytes, *bandwidth* 192 MBytes/sec dan *cache* sebesar 256 Bytes. Mikrokontroler ini juga *compatible* dengan bahasa pemrograman C++. Masukkan dari sistem *flight controller* berupa beberapa sensor pengukur data sebagai nilai yang dibutuhkan dalam sistem terutama pada sistem kendali *flying wings*. Sensor-sensor tersebut adalah *accelerometer* dan *gyroscope* yang menghasilkan data berupa nilai sudut *roll* dan *pitch* dari proses *sensor fusion* menggunakan metode madgwick quaternion serta kalman filter.

C. Desain 3D Robot *Flying Wings*

Training kit flight controller dibuat menggunakan desain model *flying wings* dengan *wingspan* sebesar 882mm dan tipe *airfoil* N-22 untuk mencapai efisiensi aerodinamika yang optimal. Model *flying wings* merupakan pesawat dengan penggabungan elemen *wing* dan *tailless* dapat memberikan performa penerbangan yang baik dan stabil. Tipe *airfoil* N-22 dipilih karena memiliki keunggulan karakteristik aerodinamika dengan performa baik pada kecepatan rendah, sehingga pesawat dapat terbang melayang (*gliding*). *Gliding* atau terbang melayang tanpa gaya dorong sendiri merupakan seni penerbangan yang mengandalkan pemahaman tentang aerodinamika, keterampilan piloting, dan pengalaman untuk mencapai performa *gliding* yang optimal. Desain 3D model pesawat *flying wings* pada *training kit flight controller* beserta dimensinya ditampilkan pada gambar 7 dan 8 berikut.



Gambar 7. Desain 3D Pesawat *Flying Wings*



Gambar 8. Dimensi Pesawat *Flying Wings*

D. Sistem Kontrol LQR (*Linear Quadratic Regulation*)

Sistem kontrol yang digunakan pada *training kit flight controller* adalah metode control yang bisa memanfaatkan waktu secara optimal dan *robust* yaitu metode LQR (*Linear Quadratic Regulation*). Metode LQR disebut linier karena model dan bentuk kontrolnya berupa linier dan disebut kuadratik karena *cost function*-nya adalah berupa kuadratik, sedangkan disebut regulator karena referensinya bukanlah berupa fungsi waktu. Persamaan sistem linier dapat dituliskan seperti persamaan berikut.

	$\dot{x} = Ax + Bu$	(1.1)
	$y = Cx + Du$	(1.2)

$\begin{bmatrix} U \\ \dot{U} \\ V \\ \dot{V} \\ W \\ \dot{W} \\ P \\ \dot{P} \\ Q \\ \dot{Q} \end{bmatrix}$	$= \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -W \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & W & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & Q & 0 & 0 & 0 & -V & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{(I_{yy}-I_{zz})}{I_{xx}} \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & \frac{I_{zz}-I_{xx}}{I_{yy}} & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} X_E \\ U \\ Y_E \\ V \\ Z_E \\ W \\ \phi \\ P \\ \theta \\ Q \end{bmatrix}$	$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ \frac{1}{m} & 0 & 0 & 0 & 0 & 0 \\ 0 & \frac{1}{m} & 0 & 0 & 0 & 0 \\ 0 & 0 & \frac{1}{m} & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{1}{m} & 0 & 0 \\ 0 & 0 & 0 & 0 & \frac{1}{I_{xx}} & 0 \\ 0 & 0 & 0 & 0 & 0 & \frac{1}{I_{yy}} \end{bmatrix}$	$\begin{bmatrix} F_x \\ F_y \\ F_z \\ L \\ M \end{bmatrix}$
$\dot{x}$	A	x	B	u
$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{bmatrix}$	$= \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$	$\begin{bmatrix} X_E \\ U \\ Y_E \\ V \\ Z_E \\ W \\ \phi \\ P \\ \theta \\ Q \end{bmatrix}$	$= \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$	$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \end{bmatrix}$
Y	C	x	D	u

Gambar 9. *State Space Autonomous Flying Wing*

Sistem kendali LQR merupakan metode kendali dengan menggunakan hukum umpan balik keadaan, persamaan umpan balik keadaan dituliskan pada persamaan berikut.

	$u = -Kx$	(1.3)
--	-----------	-------

Keterangan:

x = *State* sistem

u = *Input* proses

y = *Output* proses

A = Matriks sistem $A_{(n \times n)}$

B = Matriks sistem $B_{(m \times n)}$

C = Matriks sistem $C_{(1 \times n)}$

K = *Gain feedback*

Sistem kendali yang dirancang menjadi sistem kendali optimal, perlu dilakukan minimalisasi *cost function/quadratic function* melalui indeks performansi dalam interval $[t_0, \infty]$ seperti pada persamaan berikut.

	$J = \frac{1}{2} \int_{t_0}^{\infty} (x^T Q x + u^T R u) dt$	(1.4)
--	--	-------

Pada persamaan diatas, matriks K diperoleh dengan menggunakan persamaan sebagai berikut.

	$K = R^{-1} B^T S$	(1.5)
--	--------------------	-------

Matriks S merupakan solusi dari persamaan *riccati* seperti yang dituliskan pada persamaan berikut.

	$A^T S + SA - SBR^{-1} B^T S + Q = 0$	(1.6)
--	---------------------------------------	-------

Keterangan:

J = *cost function*

t_0 = waktu awal

∞ = waktu akhir

Q = matriks semidefinit positif

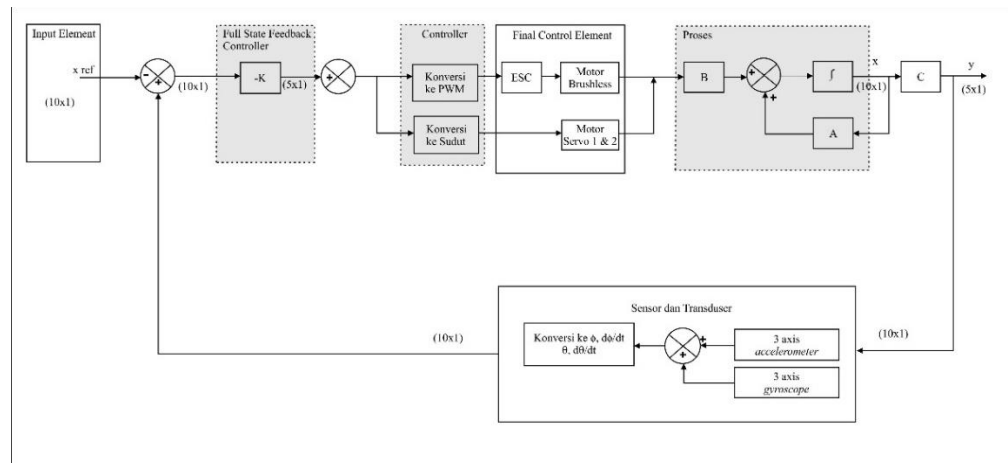
R = matriks definit positif

Dengan demikian, untuk pembobotan nilai Q dan R berpedoman pada dua hal seperti berikut.

1. Dengan semakin besar harga Q , maka akan memperbesar harga penguatan K sehingga mempercepat sistem untuk mencapai keadaan tunak (*intermediate state cost function*).
2. Dengan bertambah besar nilai R , maka akan memperkecil nilai penguatan pada K yang menyebabkan memperlambat sistem menuju pada keadaan tunak (*energy drive*).

Sistem kendali yang digunakan pada *training kit flight controller* menggunakan empat buah satte yang digunakan sebagai acuan dasar parameter kendali. Empat buah state tersebut terdiri dari:

1. Sudut *roll* (ϕ) dan kecepatan sudut *roll* (ω_ϕ)
2. Sudut *pitch* (θ) dan kecepatan sudut *pitch* (ω_θ)



Gambar 10. Diagram Blok Sistem Kendali *Flying Wings*

Keterangan:

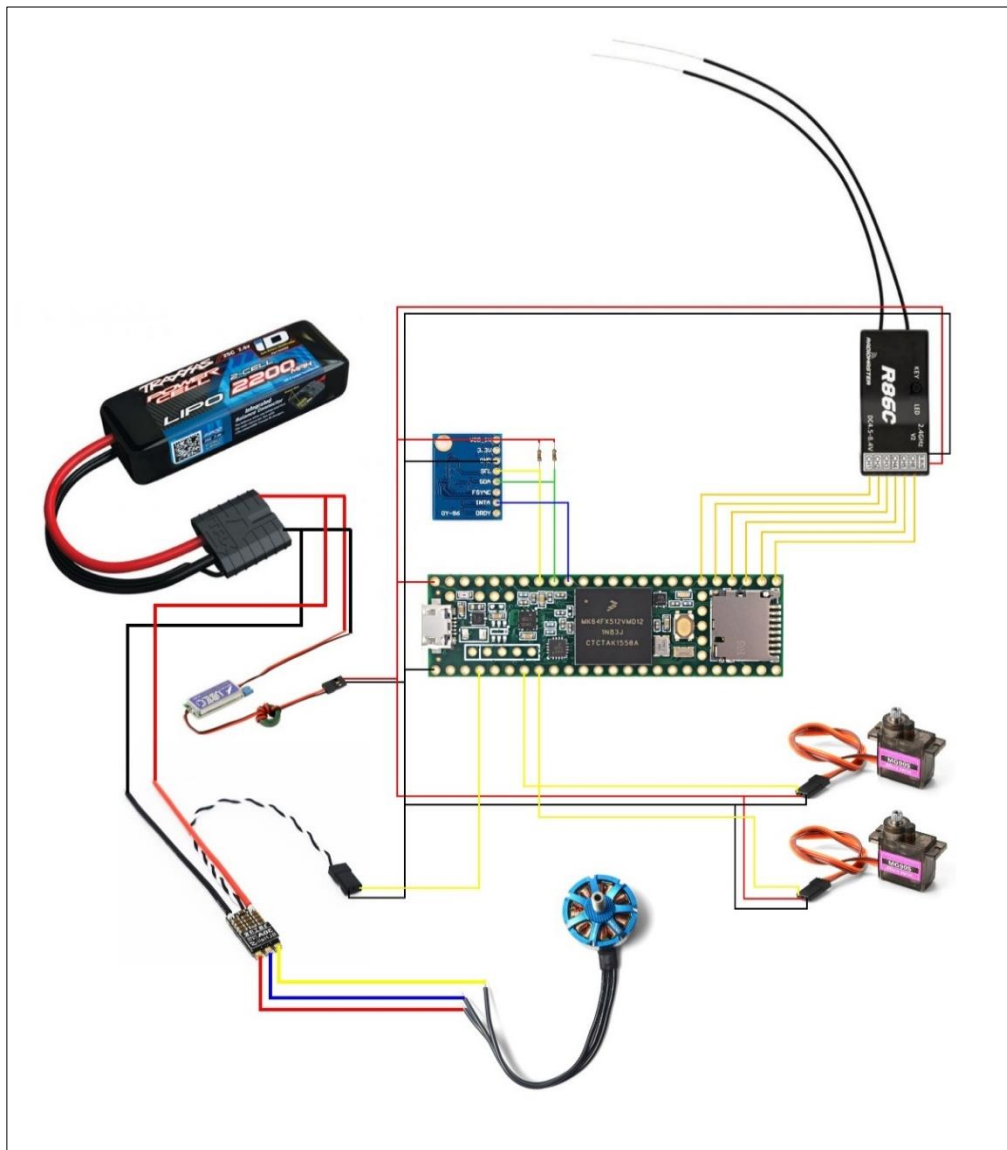
x_{Ref} = referensi <i>state</i> berupa matriks (10 x 1)	B = matriks koefisien masukan proses dengan ukuran (10 x 5)
K = <i>full state feedback gain</i> berupa matriks (5 x 10)	C = matriks koefisien keluaran proses dengan ukuran (5 x 10)
A = matriks koefisien <i>state</i> dengan ukuran (10 x 10)	y = luaran sistem untuk dipantau berupa matriks (5 x 1)

Sistem kendali pada UAV *flying-wings* untuk kendali sikap menggunakan kendali full state feedback dengan 10 state dari masukkan sensor. Hasil dari masukkan tersebut kemudian dimasukkan pada sistem *full state feedback* dengan mengalikan hasil masukkan dengan konstanta K berupa matriks (10x1). Nilai konstanta K pada sistem ini sebelumnya diolah menggunakan sistem jaringan syaraf tiruan. Sistem jaringan syaraf tiruan ini akan mengubah nilai K yang telah di tala dengan sistem LQR dengan nilai konstanta Q dan R pada setiap sumbu lalu dioperasikan dengan perhitungan Riccati. Masukkan jaringan syaraf tiruan ini sama dengan masukkan sistem pada UAV flying wing. 10 *h* Nilai u yang merupakan masukkan proses dengan kalkulasi perhitungan hasil perkalian antara gain feedback dengan state sistem yang akan dikendalikan. Sinyal masukkan kendali ini selanjutnya akan digunakan untuk pengendalian kecepatan tiap tiap rotor. Setiap rotor didefinisikan sebagai *final control element* pada sistem UAV *flying wings*.

BAB III PENGUNAAN DAN PENGOPERASIAN *TRAINING KIT FLIGHT CONTROLLER* ROBOT *FLYING WINGS* UAV

A. Wiring Diagram Training Kit Flight Controller

Wiring diagram digunakan sebagai panduan visual tentang bagaimana komponen-komponen elektronik *training kit flight controller* dihubungkan satu sama lain dengan benar, seperti yang ditampilkan pada gambar berikut.



Gambar 11. Wiring Diagram *Flight Controller* Robot *Flying Wings*

Wiring diagram tersebut merupakan salah satu bagian penting dari klarifikasi koneksi yang digunakan saat menggunakan *training kit flight*

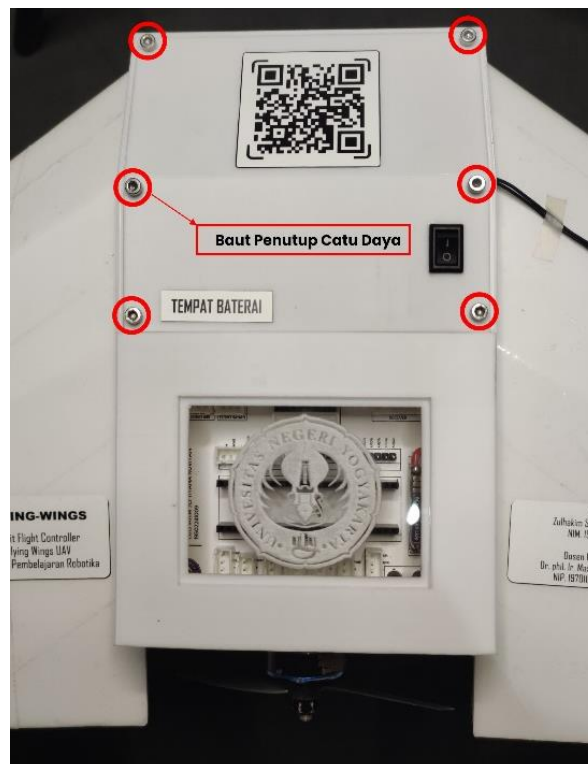
controller. Garis penghubung atau koneksi pengganti kabel antar komponen memiliki warna berbeda sesuai dengan fungsi masing-masing, dimana garis merah merupakan kabel positif/vcc (+) dan garis hitam merupakan kabel negatif/gnd (-).

B. Konfigurasi Elektronik *Training Kit Flight Controller*

Pemasangan komponen pada *training kit flight controller* dilakukan berdasarkan acuan dari wiring diagram yang dibuat. Identifikasi komponen elektronik termasuk mengenali bagian atau nama pin setiap komponen merupakan langkah awal sebelum melakukan konfigurasi dan pemasangan. Langkah-langkah pemasangan dan konfigurasi komponen dijelaskan sebagai berikut.

1. Binding Remote Kendali (*Transmitter*) Dengan *Receiver*

- Buka penutup catu daya pada *training kit flight controller* dengan melepas baut yang terpasang, sebelum melakukan konfigurasi pada remote kendali atau *transmitter*.



- b. Hidupkan remote kendali dengan menekan dan menahan tombol power yang tersedia.



- c. Buatlah model baru dengan menekan tombol garis tiga pada bagian tombol menu remote kendali.



- d. Tampilan setelah tombol garis tiga pada bagian tombol menu remote kendali akan seperti berikut.



- e. Tekan tombol ok pada bagian tombol pemilih remote kendali pilih *create model*.



- f. Tampilan setelah model baru berhasil dibuat pada remote kendali sebagai berikut.



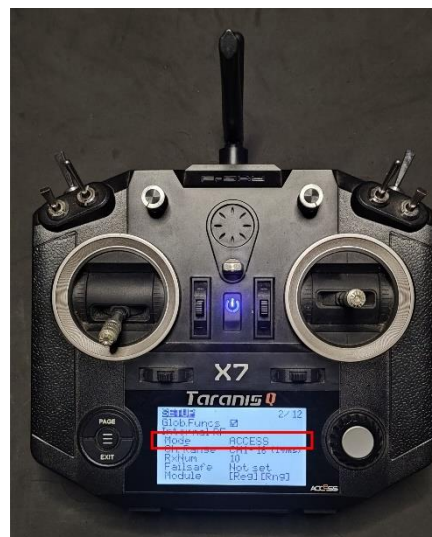
- g. Lakukan pengaturan nama pada model baru untuk mempermudah pencarian ketika digunakan, dengan menekan tombol page pada bagian tombol menu.



- h. Tekan tombol ok dan putar tombol pemilih untuk menyesuaikan nama, kemudian tekan tombol exit ketika nama sudah sesuai.



- i. Antena *transmitter* yang digunakan adalah antena external, maka matikan antena internal dengan menekan tombol ok pada bagian “mode” kemudian putar tombol pemilih sehingga berubah dari “ACCESS” menjadi “OFF”.



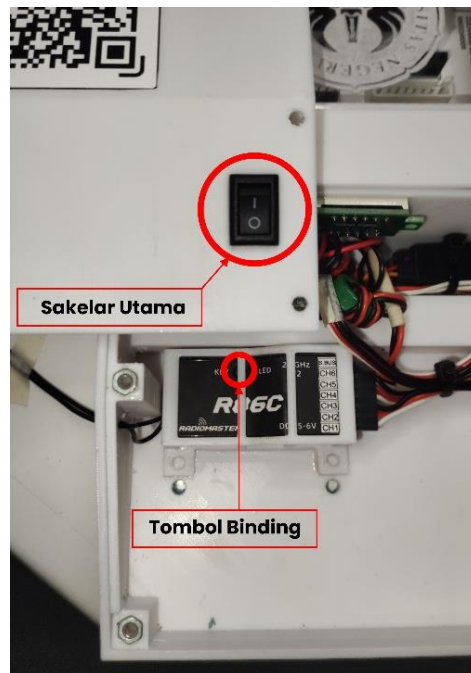
- j. Hidupkan antenna external *transmitter* dengan menekan tombol ok pada bagian “mode” kemudian putar tombol pemilih sampai pada opsi “XJT”.



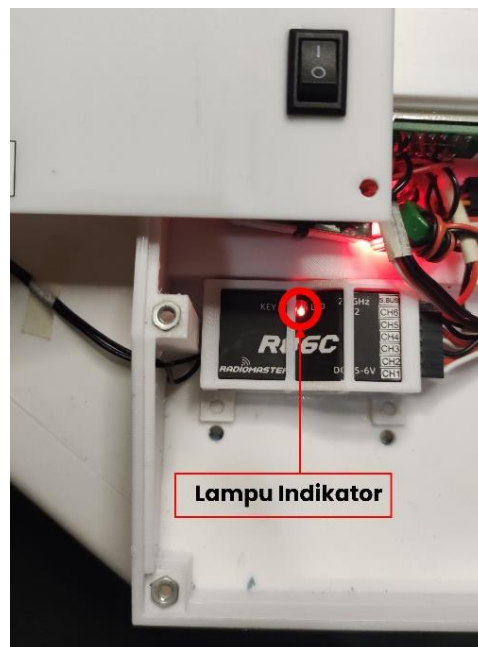
- k. Pilih bagian binding lalu tekan tombol ok, kemudian pilih opsi “Ch 1-8 telem ON” dan tekan tombol ok, maka remote kendali (*transmitter*) akan mulai mencari perangkat penerima sinyal yang aktif.



1. Setelah remote kendali (*transmitter*) dalam posisi binding, kemudian tekan tombol binding pada *receiver* bersamaan dengan menekan saklar utama pada *training kit* pada posisi “ON”.



- m. Perhatikan lampu yang menyala pada receiver, ketika lampu indikator berkedip secara cepat maka koneksi antara *transmitter* dan *receiver* berhasil, kemudian tekan tombol exit pada remote kendali (*transmitter*) dan matikan *receiver* dengan menekan saklar utama pada *training kit* pada posisi “OFF”.



- n. Hidupkan kembali *receiver* untuk memastikan koneksi antara *transmitter* dan *receiver* berhasil, yakni dengan menekan saklar utama pada *training kit*, kemudian cek pada *transmitter* jika muncul tanda sinyal seperti gambar berikut, dapat dipastikan bahwa proses binding berhasil.



2. Penambahan Channel Remote Kendali (*Transmitter*)

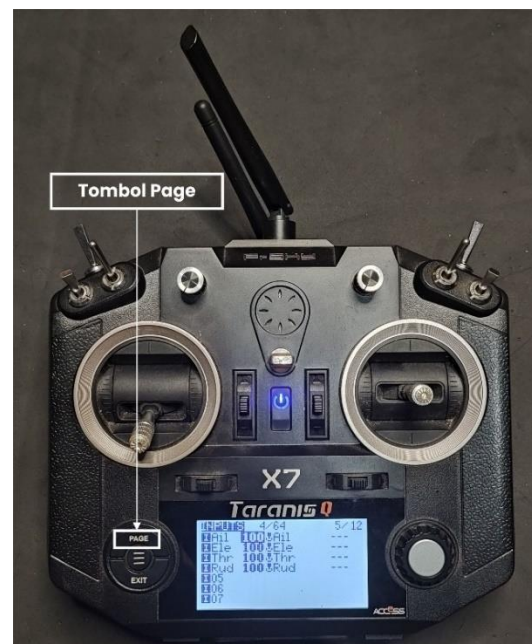
- a. Hidupkan remote kendali, kemudian tekan tombol garis tiga pada bagian tombol menu.



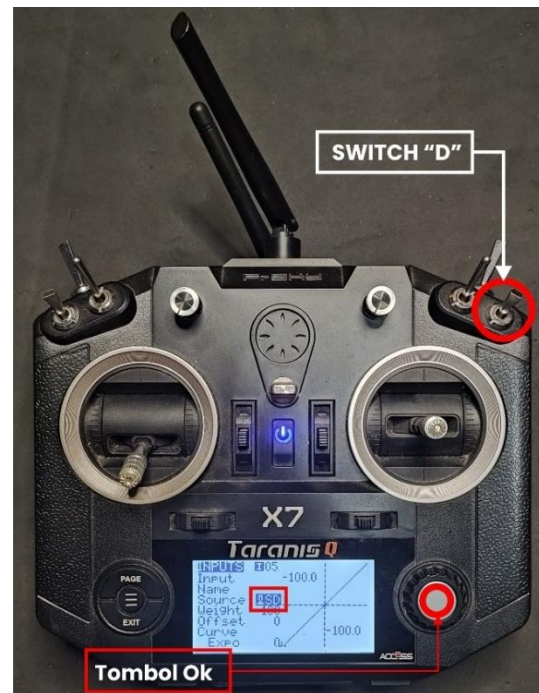
- b. Tampilan setelah tombol garis tiga pada bagian tombol menu remote kendali akan seperti berikut.



- c. Pilih model remote yang sudah kita buat sebelumnya, dalam hal ini adalah model remote “ZUL” kemudian tekan tombol page pada bagian tombol menu sampai muncul tampilan berikut.



- d. Tekan dan tahan tombol “OK” pada channel 05 untuk menambahkan CH5, kemudian pilih switch “SD” yang dalam hal ini difungsikan sebagai channel 05.



- e. Tekan dan tahan tombol “OK” pada channel 06 untuk menambahkan CH5, kemudian pilih switch “SA” yang dalam hal ini difungsikan sebagai channel 06.



- f. Tampilan ketika CH5 dan CH6 sudah berhasil ditambahkan dalam model remote kendali yang dibuat.



3. Wiring Komponen Elektronik

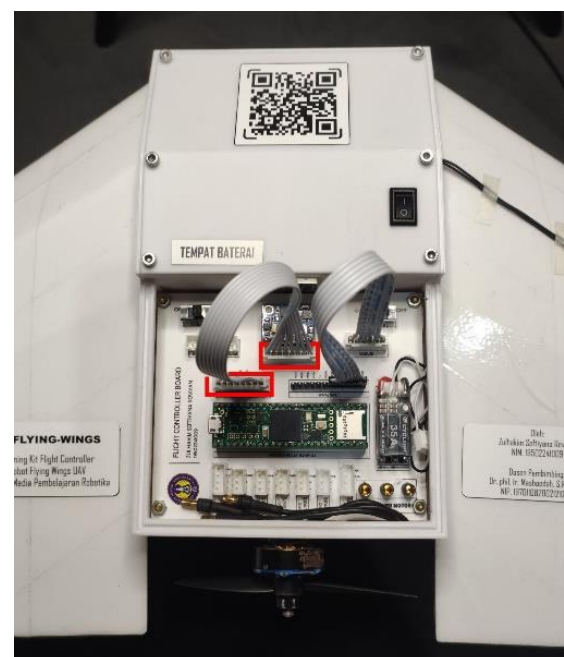
- a. Pasangkan komponen mikrokontroler Teensy 3.5 dan sensor GY-86 module pada port yang tersedia seperti pada gambar berikut.



- b. Pasang kabel konektor JST XH 6P untuk menghubungkan keluaran PWM *receiver* dengan pin input PWM mikrokontroler Teensy 3.5.



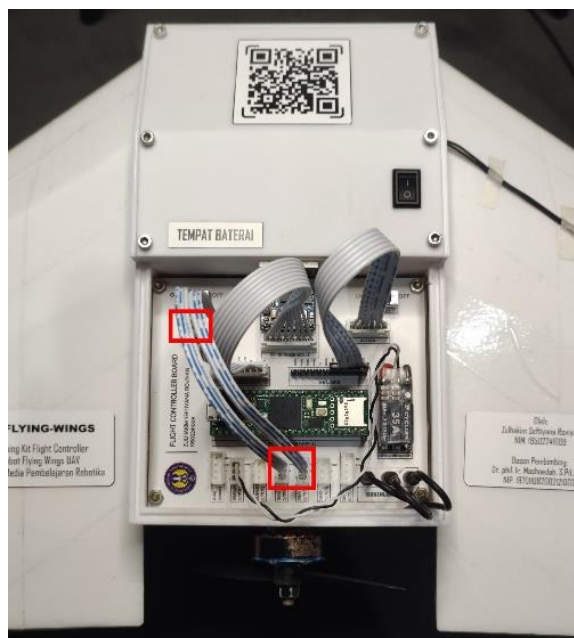
- c. Pasang kabel konektor JST XH 8P untuk menghubungkan pin sensor dengan pin mikrokontroler Teensy 3.5.



- d. Pasang kabel PWM ESC pada *socket brushless output* 1 dan kabel 3-phase motor BLDC pada *socket brushless motor* seperti pada gambar berikut.



- e. Pasang kabel konektor JST XH 3P untuk menghubungkan servo dengan output PWM mikrokontroler, yakni *socket output* motor servo 1 dengan *socket* servo kiri dan *socket output* motor servo 2 dengan *socket* servo kanan.



- f. Hidupkan *training kit flight controller* dengan menekan saklar utama, kemudian aktifkan ESC serta mikrokontroler Teensy 3.5 dengan menekan saklar 1 dan saklar 2 pada papan komponen elektronik.



- g. Tampilan ketika *training kit flight controller* hidup divualisasikan seperti gambar berikut, dimana minivoltmeter dalam penggunaan *training kit flight controller* menjadi indikator nilai kapasitas baterai yang digunakan. Kapasitas baterai yang disarankan dalam penggunaan *training kit flight controller* adalah pada kapasitas minimum 7,4 volt dari maksimum baterai 2 cell yaitu 8,4 volt. Ketika indikator baterai menunjukkan angka dibawah 7,4 volt maka baterai harus dilakukan pengisian ulang agar kualitas baterai tidak menurun tetap pada performa yang maksimal.



C. Pemrograman *Training Kit Flight Controller*

1. *Software Arduino IDE*

Arduino IDE (*Integrated Development Environment*) merupakan sebuah perangkat lunak yang digunakan untuk memprogram dan mengembangkan aplikasi untuk *board* mikrokontroler Arduino dan sejenisnya. *Software* ini menyediakan lingkungan yang lengkap untuk membuat, mengedit, mengkompilasi, dan mengunggah program ke dalam *board* arduino. Arduino IDE termasuk salah satu perangkat lunak *open-source* yang dapat diunduh secara gratis dalam pengembangan sebuah proyek dengan berbagai mikrokontroler yang *compatible*. Arduino IDE memiliki editor teks yang merupakan tempat untuk menulis dan mengedit kode program dalam bahasa pemrograman arduino. *Sketch* merupakan nama dari sebuah program yang dibuat dalam Arduino IDE, dimana setiap *sketch* terdiri dari dua fungsi yaitu “*setup*” dan “*loop*”. Fungsi “*setup*” dieksekusi sekali saat mikrokontroler digunakan saat menginisialisasi

variabel dan pengaturan awal, sedangkan fungsi “*loop*” dilaksanakan berulang kali selama mikrokontroler aktif. Arduino IDE dilengkapi dengan kompilator (*compiler*) yang mengubah kode bahasa pemrograman menjadi kode mesin (*binary*) yang dapat dijalankan oleh mikrokontroler. Arduino IDE juga menyediakan fitur *uploader* yang memungkinkan pengguna untuk mengunggah program yang telah dilakukan kompilasi ke mikrokontroler melalui koneksi USB. Serial monitor menjadi fitur dari Arduino IDE yang digunakan untuk membaca data yang dikirimkan oleh mikrokontroler. Arduino IDE menyediakan banyak perpustakaan (*libraries*) yang berisikan kode yang telah dikompilasi sebelumnya untuk berbagai perangkat atau sensor. Fitur *libraries* ini mempermudah pengguna untuk mengakses berbagai fungsi dari komponen dengan lebih mudah tanpa harus menulis kode dari awal. *Tools* atau alat merupakan fitur lain dari Arduino IDE yang berisikan berbagai alat dan pengaturan untuk memenuhi kebutuhan dalam penggunaan *software*. Arduino IDE memiliki keunggulan yakni dilengkapi dengan contoh program (*example sketches*) yang memuat berbagai kode program untuk mengakses sensor, aktuator, dan komponen lain. Arduino IDE juga memiliki kompatibilitas dengan berbagai *board* mikrokontroler seperti Arduino Uno, Arduino Nano, Arduino Pro-Mini, ESP-32, ESP-8266, Teensy, STM-32, dan mikrokontroler lain untuk pengembangan proyek elektronika serta robotika.

2. Bahasa Pemrograman Arduino IDE

Bahasa pemrograman C/C++ merupakan basis bahasa yang digunakan pada Arduino IDE dengan beberapa penyederhanaan untuk mempermudah dalam membuat kode program. Bahasa pemrograman dalam Arduino IDE relatif mudah dipahami dan didukung oleh penggunaan *libraries* yang disediakan, sehingga dapat dengan mudah diimplementasikan dalam berbagai jenis pengembangan proyek elektronik dan robotika. Berikut merupakan uraian secara detail tentang bahasa pemrograman Arduino IDE.

- a. Setiap program pada Arduino IDE terdiri dari dua fungsi utama yakni “setup()” dan “loop()”. Fungsi “setup()” dieksekusi sekali saat mikrokontroler diaktifkan, dimana fungsi ini digunakan untuk melakukan inisialisasi variabel, pin, dan pengaturan awal pada program. Fungsi “loop()” dieksekusi berulang kali selama mikrokontroler aktif, dimana kode ini biasa digunakan untuk akses komponen seperti sensor dan aktuator.
- b. Variabel pada pemrograman Arduino IDE dapat dideklarasikan dengan berbagai tipe data seperti int (integer), float (floating-point number), char (karakter), dan tipe data lain.
- c. Arduino IDE mendukung bahasa pemrograman struktur kontrol seperti if-else (percabangan), for (pengulangan), while (pengulangan dengan kondisi), dan do-while (pengulangan minimal satu kali dengan kondisi).
- d. Fungsi pada bahasa pemrograman Arduino IDE dapat dideklarasikan dengan berbagai tipe fungsi seperti fungsi matematika “abs(), pow(), sqrt(), map(), dan lain-lain”, fungsi waktu “milis(), micros()” serta fungsi lain untuk pengaturan PWM (*Pulse Width Modulation*), interrupt, komunikasi I2C, SPI, dan lain sebagainya.
- e. Komunikasi serial disediakan Arduino IDE untuk melakukan komunikasi mikrokontroler dengan perangkat eksternal, menggunakan bahasa pemrograman seperti “Serial.begin()”, “Serial.println”, dan “Serial.read()”.
- f. Arduino IDE memiliki bahasa pemrograman yang memiliki fungsi untuk mengatur mode pin sebagai input atau output menggunakan “pinMode()”. Selain itu, fungsi “digitalWrite()” dimiliki oleh Arduino IDE untuk mengirimkan sinyal HIGH atau LOW ke output, sedangkan “digitalRead()” digunakan untuk membaca nilai digital dari pin input.

3. Kode Program *Training Kit Flight Controller*

Pemrograman *training kit flight controller* dilakukan dengan menggunakan Arduino IDE yang dapat diunduh pada laman: <https://www.arduino.cc/en/software>. *Libraries* yang digunakan dapat diunduh pada laman berikut: <https://bit.ly/3O89Xtm>. Kode program yang digunakan terdiri dari beberapa bagian seperti berikut.

Program Utama

```
//===== HEADER LIBRARY =====
//MPU
#include "Wire.h"
#include "I2Cdev.h"
#include "MPU6050_6Axis_MotionApps20.h"
#include "math.h"

//MOTOR
#include "Servo.h"

//MAGNETO
#include <HMC5883L.h>

//===== IMU =====
MPU6050 mpu;
MPU6050 accelgyro;
int16_t mx, my, mz;
int16_t ax, ay, az;
int16_t gx, gy, gz;
float gyroX_filt, gyroY_filt, gyroZ_filt;

float G_Dt      = 0.005;
float trim_yaw   = 2;
float gain_roll  = 2;
float gain_pitch = 2;
float gain_vel_roll = 0.005;
float gain_vel_pitch = 0.005;
float gain_vel_yaw = 0.010;
float gain_vel_z   = 0;

double rad_yaw, rad_pitch, rad_roll;
double roll_kalman, pitch_kalman, yaw_kalman;
```

```
double accum_roll = 0;
double accum_pitch = 0;
double k_acc = 0;
double k_gps = 0;
double yaw_deg;
double pitch_deg;
double roll_deg;
double pitch_deg_previous, roll_deg_previous;

#define OUTPUT_READABLE_YAWPITCHROLL
#define OUTPUT_READABLE_ACCELGYRO

bool dmpReady = false;
uint8_t mpuIntStatus;
uint8_t devStatus;
uint16_t packetSize;
uint16_t fifoCount;
uint8_t fifoBuffer[64];

Quaternion q;
VectorFloat gravity;
VectorInt16 aa;
VectorInt16 aaReal;
VectorInt16 aaWorld;
float euler[3];
float ypr[3];

volatile bool mpuInterrupt = false;
boolean interruptLock = false;

void dmpDataReady(){ mpuInterrupt = true; }

//===== REMOTE
=====
volatile int channel1 = 0;
volatile int roll_channel = 0;
volatile int channel2 = 0;
volatile int throttle_channel = 0;
volatile int channel3 = 0;
volatile int pitch_channel = 0;
volatile int channel4 = 0;
volatile int yaw_channel = 0;
volatile int channel5 = 0;
volatile int ch5_channel = 0;
volatile int channel6 = 0;
volatile int ch6_channel = 0;

int ch5, ch6, throttle, throttle_input, heading_mode,
heading_model;
int roll_input, pitch_input, yaw_input;
char inChar;
```

```
//===== MOTOR
=====
#define MOTOR1 2
Servo brushless1;
Servo myservoY; //roll
Servo myservoX; //pitch
int motor1;

//===== SERVO
=====
int servoAngleInit1 = 87;
int servoAngleInit2 = 78;

int servo1_up      = 35;
int servo1_down    = 120;
int servo2_up      = 42;
int servo2_down    = 132;

int servo1_upauto  = 37;
int servo1_downauto = 122;
int servo2_upauto  = 42;
int servo2_downauto = 132;

int Servo1, Servo2;
float      rollControlServo,      pitchControlServo,
yawControlServo;

//=====
COMPASS =====
HMC5883L compass;
float headingDegrees;
float heading, setHeading;
float heading_reference;
float heading_control;
float acc_x, acc_y;
float compensateRoll, compensatePitch;
float cosComRoll, sinComRoll, cosComPitch, sinComPitch;
float Yh, Xh, Ymag_correct, Xmag_correct;

//=====BAROME
TER=====
uint8_t baro_error_code;
int  b, countBaro = 0, barSumCount = 0;
bool  received_D1_conversion = false;
float pressure_mbar = 0.0, temperature_c = 0.0;
const int pressure_mbar_history_length = 4;
float
pressure_mbar_history[pressure_mbar_history_length];
float pressure_mbar_history_sum, pressure_mbar_average;
```

```

float      accel_z,      AltitudeBaroGround,      accelZ_init,
estimation_altitude, estimation_vel_z;
float      z_position,   z_velocity,   alt_ref,   accelZ_ref,
altitudeControl, altVelControl;
float      altitudeControl_ref,          altVelControl_ref,
AltitudeControl;
int axis = 0;

float altitude;
float altitude_set, altitude_control, altitude_gain = 50;
float realketinggian;

//===== TIMER
=====
unsigned long timerGPS;
unsigned long timerGPS2;
unsigned long timerXV;
unsigned      long          waypointTime_current,
waypointTime_previous;
unsigned long currentTime_compass, previousTime_compass;
unsigned      long          currentTime_barometer,
previousTime_barometer;
unsigned long currentTime_GPS, previousTime_GPS;
unsigned long currentTime_gps, previousTime_gps;

uint32_t timeProgram, previousTimeProgram;
uint32_t timeServo, previousTimeServo;

int index_condition; // 0 = manual, 1 = auto
int GPS_time;
int GPS2_time;
int XV_time;

void setup()
{
    #if I2CDEV_IMPLEMENTATION == I2CDEV_ARDUINO_WIRE
        Wire.begin();
    #elif I2CDEV_IMPLEMENTATION == I2CDEV_BUILTIN_FASTWIRE
        Fastwire::setup(400, true);
    #endif

    myservoX.attach(5);
    myservoY.attach(6);

    Serial.begin(115200);
    init_MPU();
    compass_init();
    remote_init();
    delay(50);

    servo_init();

```

```
motor_setup();
}

void loop()
{
    get_YPR();
    compass_update();
    mapremote();

    if (ch5 == 1 && ch6 == 0)
    {
        throttle          = throttle_channel;
        throttle_input    = throttle_channel;
        controlUpdateManual();
        index_condition = 0;
        setHeading        = headingDegrees;
        // init_Home      = 0; waypoint_index = 0; tracking
= 0; cek_heading = 0; bearing = 0;
    }

    if (ch5 == 1 && ch6 == 1)
    {
        throttle = throttle_channel;
        controlUpdateAuto();
        index_condition = 1;
    }

    gerak_servo();
    update_motor();
    timeProgram = micros();
    if (timeProgram - previousTimeProgram >= 100000)
    {
        print_out();
        previousTimeProgram = micros();
    }
}
```

Kode Program: Fungsi Remote

```
void remote_init()
{
    pinMode(33, INPUT_PULLUP);
    pinMode(34, INPUT_PULLUP);
    pinMode(35, INPUT_PULLUP);
    pinMode(36, INPUT_PULLUP);
    pinMode(37, INPUT_PULLUP);
    pinMode(38, INPUT_PULLUP);

    attachInterrupt(33, rollris, RISING);
    attachInterrupt(34, pitchris, RISING);
    attachInterrupt(35, throtrris, RISING);
    attachInterrupt(36, yawris, RISING);
    attachInterrupt(37, ch5ris, RISING);
    attachInterrupt(38, ch6ris, RISING);
}

//===== roll channel =====
void rollris()
{
    attachInterrupt(33, rollfall, FALLING);
    channel3 = micros();
}

void rollfall()
{
    attachInterrupt(33, rollris, RISING);
    roll_channel = micros()-channel3;
}

//===== pitch channel =====
void pitchris()
{
    attachInterrupt(34, pitchfall, FALLING);
    channel1 = micros();
}

void pitchfall()
{
    attachInterrupt(34, pitchris, RISING);
    pitch_channel = micros()-channel1;
}

//===== throttle channel =====
void throtrris()
{
    attachInterrupt(35, throtfall, FALLING);
    channel2 = micros();
}
```

```
void throtfall()
{
    attachInterrupt(35, throtrise, RISING);
    throttle_channel = micros()-channel2;
}

//===== yaw channel =====
void yawrise()
{
    attachInterrupt(36, yawfall, FALLING);
    channel4 = micros();
}

void yawfall()
{
    attachInterrupt(36, yawrise, RISING);
    yaw_channel = micros()-channel4;
}

//===== ch5 channel =====
void ch5rise()
{
    attachInterrupt(37, ch5fall, FALLING);
    channel5 = micros();
}

void ch5fall()
{
    attachInterrupt(37, ch5rise, RISING);
    ch5_channel = micros()-channel5;
}

//===== ch6 channel =====
void ch6rise()
{
    attachInterrupt(38, ch6fall, FALLING);
    channel6 = micros();
}

void ch6fall()
{
    attachInterrupt(38, ch6rise, RISING);
    ch6_channel = micros()-channel6;
}

void mapremote()
{
    roll_input = 0;
    if (roll_channel > 1514){roll_input=roll_channel-
1514;}else if(roll_channel <
```

```

1504){roll_input=roll_channel-1504;} roll_input
=constrain(roll_input,-505,497); roll_input /= 12.625;
pitch_input = 0;
if (pitch_channel > 1504){pitch_input=pitch_channel-
1504;}else if(pitch_channel <
1494){pitch_input=pitch_channel-1494;} pitch_input
=constrain(pitch_input,-507,507); pitch_input /= 12.675;
yaw_input = 0;
if (yaw_channel > 1507){yaw_input=yaw_channel-
1507;}else if(yaw_channel < 1497){yaw_input=yaw_channel-
1497;} yaw_input =constrain(yaw_input,-510,504);
yaw_input /= 12.75;

if(ch5_channel<1400){ch5=0;}if(ch5_channel>1400){ch5=1;}

if(ch6_channel<1400){ch6=0;}if(ch6_channel>1400){ch6=1;}
if(ch6_channel<1800){heading_mode=0;}
if(ch6_channel>1800){heading_mode=1;}
}

```

Kode Program : Fungsi IMU

```

void init_MPU() {
    mpu.initialize();
    devStatus = mpu.dmpInitialize();

    mpu.setXGyroOffset(-5);
    mpu.setYGyroOffset(32);
    mpu.setZGyroOffset(-4);
    mpu.setXAccelOffset(1382);
    mpu.setYAccelOffset(1828);
    mpu.setZAccelOffset(702);
    if (devStatus == 0) {

        mpu.setDMPEnabled(true);
        attachInterrupt(17, dmpDataReady, RISING);
        mpuIntStatus = mpu.getIntStatus();
        packetSize = mpu.dmpGetFIFOPacketSize();
    }

    mpu.setI2CMasterModeEnabled(false);
    mpu.setI2CBypassEnabled(false);
    mpu.setSleepEnabled(false);
}

void get_YPR() {
    accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
    gyroX_filt = gyroX_filt * 0.93 + gx * 0.07;
    gyroY_filt = (gyroY_filt * 0.93 + gy * 0.07) * -1;
    gyroZ_filt = gyroZ_filt * 0.93 + gz * 0.07;
    mpuInterrupt = false;
}

```

```

mpuIntStatus = mpu.getIntStatus();
fifoCount = mpu.getFIFOCount();
if ((mpuIntStatus & 0x10) || fifoCount >= 1024) {
    mpu.resetFIFO();
} else if (mpuIntStatus & 0x02) {
    while (fifoCount < packetSize) fifoCount =
mpu.getFIFOCount();
    mpu.getFIFOBytes(fifoBuffer, packetSize);
    fifoCount -= packetSize;

    mpu.dmpGetQuaternion(&q, fifoBuffer);
    mpu.dmpGetGravity(&gravity, &q);
    mpu.dmpGetYawPitchRoll(ypr, &q, &gravity);
    pitch_deg = (ypr[1] * 180 / M_PI)+2.67;
    roll_deg = (ypr[2] * 180 / M_PI)+55.37;
    yaw_deg=(ypr[0] * 180/M_PI)+0.32;

    mpu.dmpGetQuaternion(&q, fifoBuffer);
    mpu.dmpGetAccel(&aa, fifoBuffer);
    mpu.dmpGetGravity(&gravity, &q);
    mpu.dmpGetLinearAccel(&aaReal, &aa, &gravity);
    mpu.dmpGetLinearAccelInWorld(&aaWorld, &aaReal, &q);

}
}

```

Kode Program : Fungsi Compass

```

void compass_init()
{
    mpu.setI2CMasterModeEnabled(false);
    mpu.setI2CBypassEnabled(true);
    mpu.setSleepEnabled(false);

    while (!compass.begin())
    {
        Serial.println("Could not find a valid HMC5883L
sensor, check wiring!");
        delay(500);
    }

    compass.setRange(HMC5883L_RANGE_1_3GA);
    compass.setMeasurementMode(HMC5883L_CONTINUOUS);
    compass.setDataRate(HMC5883L_DATARATE_30HZ);
    compass.setSamples(HMC5883L_SAMPLES_8);
    compass.setOffset(0, 0);
}

void compass_update()
{
    compass_compentation();
}

```

```

float heading = atan2(Yh, Xh);
float declinationAngle = (9.0 + (26.0 / 60.0)) / (180 /
M_PI);
heading += declinationAngle;

if (heading < 0)
{
    heading += 2 * PI;
}

if (heading > 2 * PI)
{
    heading -= 2 * PI;
}

headingDegrees = (heading * 180/M_PI);

heading_reference = setHeading - headingDegrees;
if(heading_reference > 180) { heading_reference -=
360;}
if(heading_reference < - 180) { heading_reference +=
360;}
heading_control = heading_reference;
}

void compass_compentation()
{
    Vector norm = compass.readNormalize();
    acc_x = ax*2.0 / 32768.0;
    acc_y = ay*2.0 / 32768.0;

    acc_x = constrain(acc_x, -1, 1);
    acc_y = constrain(acc_y, -1, 1);

    compensatePitch = asin(-acc_x);
    compensateRoll = asin(acc_y);

    cosComRoll = cos(compensateRoll);
    sinComRoll = sin(compensateRoll);
    cosComPitch = cos(compensatePitch);
    sinComPitch = sin(compensatePitch);

    Yh = norm.YAxis*cosComRoll + norm.ZAxis*sinComRoll;
    Xh = norm.XAxis*cosComPitch +
norm.YAxis*sinComRoll*sinComPitch -
norm.ZAxis*cosComRoll*sinComPitch;
}

```

Kode Program : Fungsi Motor BLDC

```
void motor_setup()
{
    pinMode(MOTOR1, OUTPUT);
    brushless1.attach(MOTOR1);
    brushless1.writeMicroseconds(1000);
    delay(1000);
}

void update_motor()
{
    if(ch5==0)
    {
        Servo1 = servoAngleInit1;
        Servo2 = servoAngleInit2;
        motor1 = 1000;
        brushless1.writeMicroseconds(motor1);
    }

    if(ch5==1)
    {
        motor1=throttle;
        brushless1.writeMicroseconds(motor1);
    }
}
```

Kode Program: Fungsi Motor Servo

```
void gerak_servo()
{
    timeServo = micros();
    if(timeServo - previousTimeServo >= 10000)
    {
        myservoX.write(Servo1);
        myservoY.write(Servo2);
        previousTimeServo = micros();
    }
}

void servo_init()
{
    myservoX.write(servoAngleInit1);
    myservoY.write(servoAngleInit2);
}

void controlUpdateAuto()
{
    rollControlServo = (((roll_deg*-1) +
(roll_input))*gain_roll) + (gyroX_filt*gain_vel_roll);
```

```

    pitchControlServo = (((pitch_deg*-1) + (pitch_input*-1))*gain_pitch) + (gyroY_filt*gain_vel_pitch);
    rollControlServo   = constrain(rollControlServo, -30,30);
    pitchControlServo = constrain(pitchControlServo, -30,30);

    if(heading_model == 0 && ch5 == 1 && ch6 == 1) {throttle = throttle_channel; setHeading = headingDegrees; yawControlServo = 0 ;}
    if(heading_model == 1 && heading_mode == 1 && ch5 == 1 && ch6 == 1){throttle = throttle_channel; yawControlServo = (yaw_input + heading_control*trim_yaw) + (gyroZ_filt*gain_vel_yaw); yawControlServo = constrain(yawControlServo, -30, 30);}

    Servo1 = (pitchControlServo) + (rollControlServo*-1) + (yawControlServo*-1) ; Servo1 = constrain(Servo1, -30,30);
    Servo1 = map(Servo1,-30,30,servo1_downauto,servo1_upauto);
    Servo2 = (pitchControlServo*-1) + (rollControlServo*-1) + (yawControlServo*-1) ; Servo2 = constrain(Servo2, -30,30);
    Servo2 = map(Servo2,-30,30,servo2_downauto,servo2_upauto);

}

void controlUpdateManual()
{
    Servo1 = (roll_input*-1) + (pitch_input*-1); Servo1 = constrain(Servo1, -30,30); Servo1 = map(Servo1,-30,30,servo1_down,servo1_up);
    Servo2 = (roll_input*-1) - (pitch_input*-1); Servo2 = constrain(Servo2, -30,30); Servo2 = map(Servo2,-30,30,servo2_down,servo2_up);
}

```

Kode Program: Fungsi Serial Display

```

void print_out(){
    timeProgram = micros();
    if (timeProgram - previousTimeProgram >= 100000)
    {
        // Serial.print("altitude : ");
        // Serial.print(altitude);
        // Serial.print("\t altitude real : ");
        // Serial.print(realketinggian);
        // Serial.print(throttle_channel) ;Serial.print("\t");
        // Serial.print(pitch_channel) ;Serial.print("\t");
        // Serial.print(roll_channel) ;Serial.print("\t");
        // Serial.print(yaw_channel) ;Serial.print("\t");
    }
}

```

```
// Serial.print(ch5_channel)      ;Serial.print("\t");
// Serial.print(ch6_channel)      ;Serial.print("\t");
// Serial.print(ch7_channel)      ;Serial.print("\t");
// Serial.print(ch8_channel)      ;Serial.print("\t");
// Serial.print(ch9_channel)      ;Serial.print("\t");
// Serial.print(heading_mode)     ;Serial.print("\t");
// Serial.print(heading_model)    ;Serial.print("\t");
atitude_data();
// motorServo_AutoTrimming();
// gainTuningFeedback();
Serial.println();
previousTimeProgram = micros();
}}

void attitude_data()
{
  Serial.print(roll_deg)          ;Serial.print("\t");
  Serial.print(pitch_deg)         ;Serial.print("\t");
  Serial.print(yaw_deg)           ;Serial.print("\t");
  Serial.print(headingDegrees)    ;Serial.print("\t");
}
```

4. Kalibrasi Sensor IMU

Kalibrasi sensor IMU MPU6050 (*accelerometer* dan *gyroscope*) merupakan langkah yang penting dalam memastikan data yang dihasilkan oleh sensor akurat dan dapat diandalkan. Kalibrasi sensor IMU dilakukan dengan mencari nilai “*offset*”, dimana “*offset*” sendiri merupakan perbedaan antara nilai yang diukur oleh sensor dengan nilai yang seharusnya mewakili nol. Nilai “*offset*” dicari dengan menggunakan kode program berikut.

Kalibrasi Sensor IMU

```
// I2Cdev and MPU6050 must be installed as libraries
#include "I2Cdev.h"
#include "MPU6050.h"
#include "Wire.h"

//////////////////////////////////////             CONFIGURATION
//////////////////////////////////////

//Change this 3 variables if you want to fine tune the
sketch to your needs.
```

```
int buffersize=1000;          //Amount of readings used to
average, make it higher to get more precision but sketch
will be slower (default:1000)
int acel_deadzone=8;          //Acelerometer error allowed,
make it lower to get more precision, but sketch may not
converge (default:8)
int giro_deadzone=1;          //Giro error allowed, make it
lower to get more precision, but sketch may not converge
(default:1)

// default I2C address is 0x68
// specific I2C addresses may be passed as a parameter
here
// AD0 low = 0x68 (default for InvenSense evaluation
board)
// AD0 high = 0x69
//MPU6050 accelgyro;
MPU6050 accelgyro(0x68); // <-- use for AD0 high

int16_t ax, ay, az,gx, gy, gz;

int
mean_ax,mean_ay,mean_az,mean_gx,mean_gy,mean_gz,state=0;
int
ax_offset,ay_offset,az_offset,gx_offset,gy_offset,gz_off
set;

//////////////////////////////////////          SETUP
//////////////////////////////////////
void setup() {
  // join I2C bus (I2Cdev library doesn't do this
automatically)
  Wire.begin();
  // COMMENT NEXT LINE IF YOU ARE USING ARDUINO DUE
  TWBR = 24; // 400kHz I2C clock (200kHz if CPU is 8MHz).
Leonardo measured 250kHz.

  // initialize serial communication
  Serial.begin(115200);
```

```
// initialize device
accelgyro.initialize();

// wait for ready
while (Serial.available() && Serial.read()); // empty
buffer
while (!Serial.available()){
    Serial.println(F("Send any character to start
sketch.\n"));
    delay(1500);
}
while (Serial.available() && Serial.read()); // empty
buffer again

// start message
Serial.println("\nMPU6050 Calibration Sketch");
delay(2000);
Serial.println("\nYour MPU6050 should be placed in
horizontal position, with package letters facing up.
\nDon't touch it until you see a finish message.\n");
delay(3000);
// verify connection
Serial.println(accelgyro.testConnection() ? "MPU6050
connection successful" : "MPU6050 connection failed");
delay(1000);
// reset offsets
accelgyro.setXAccelOffset(0);
accelgyro.setYAccelOffset(0);
accelgyro.setZAccelOffset(0);
accelgyro.setXGyroOffset(0);
accelgyro.setYGyroOffset(0);
accelgyro.setZGyroOffset(0);
}

////////////////////////////////////// LOOP
//////////////////////////////////////
void loop() {
    if (state==0){
```

```
Serial.println("\nReading sensors for first
time...");
meansensors();
state++;
delay(1000);
}

if (state==1) {
Serial.println("\nCalculating offsets...");
calibration();
state++;
delay(1000);
}

if (state==2) {
meansensors();
Serial.println("\nFINISHED!");
Serial.print("\nSensor readings with offsets:\t");
Serial.print(mean_ax);
Serial.print("\t");
Serial.print(mean_ay);
Serial.print("\t");
Serial.print(mean_az);
Serial.print("\t");
Serial.print(mean_gx);
Serial.print("\t");
Serial.print(mean_gy);
Serial.print("\t");
Serial.println(mean_gz);
Serial.print("Your offsets:\t");
Serial.print(ax_offset);
Serial.print("\t");
Serial.print(ay_offset);
Serial.print("\t");
Serial.print(az_offset);
Serial.print("\t");
Serial.print(gx_offset);
Serial.print("\t");
Serial.print(gy_offset);
```

```

Serial.print("\t");
Serial.println(gz_offset);
Serial.println("\nData is printed as: acelX acely
acelZ giroX giroY giroZ");
Serial.println("Check that your sensor readings are
close to 0 0 16384 0 0 0");
Serial.println("If calibration was succesful write
down your offsets so you can set them in your projects
using something similar to
mpu.setXAccelOffset(youroffset)");
while (1);
}
}

////////////////////////////////////// FUNCTIONS
//////////////////////////////////////

void meansensors() {
    long
i=0,buff_ax=0,buff_ay=0,buff_az=0,buff_gx=0,buff_gy=0,bu
ff_gz=0;

    while (i<(buffersize+101)){
        // read raw accel/gyro measurements from device
        accelgyro.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

        if (i>100 && i<=(buffersize+100)){ //First 100
measures are discarded
            buff_ax=buff_ax+ax;
            buff_ay=buff_ay+ay;
            buff_az=buff_az+az;
            buff_gx=buff_gx+gx;
            buff_gy=buff_gy+gy;
            buff_gz=buff_gz+gz;
        }
        if (i==(buffersize+100)){
            mean_ax=buff_ax/buffersize;
            mean_ay=buff_ay/buffersize;
            mean_az=buff_az/buffersize;
            mean_gx=buff_gx/buffersize;

```

```
        mean_gy=buff_gy/buffersize;
        mean_gz=buff_gz/buffersize;
    }
    i++;
    delay(2); //Needed so we don't get repeated measures
}
}

void calibration(){
    ax_offset=-mean_ax/8;
    ay_offset=-mean_ay/8;
    az_offset=(16384-mean_az)/8;

    gx_offset=-mean_gx/4;
    gy_offset=-mean_gy/4;
    gz_offset=-mean_gz/4;
    while (1){
        int ready=0;
        accelgyro.setXAccelOffset(ax_offset);
        accelgyro.setYAccelOffset(ay_offset);
        accelgyro.setZAccelOffset(az_offset);

        accelgyro.setXGyroOffset(gx_offset);
        accelgyro.setYGyroOffset(gy_offset);
        accelgyro.setZGyroOffset(gz_offset);

        meansensors();
        Serial.println("...");

        if (abs(mean_ax)<=acel_deadzone) ready++;
        else ax_offset=ax_offset-mean_ax/acel_deadzone;

        if (abs(mean_ay)<=acel_deadzone) ready++;
        else ay_offset=ay_offset-mean_ay/acel_deadzone;

        if (abs(16384-mean_az)<=acel_deadzone) ready++;
        else
            az_offset=az_offset+(16384-
mean_az)/acel_deadzone;
```

```
if (abs(mean_gx)<=giro_deadzone) ready++;  
else gx_offset=gx_offset-mean_gx/(giro_deadzone+1);  
  
if (abs(mean_gy)<=giro_deadzone) ready++;  
else gy_offset=gy_offset-mean_gy/(giro_deadzone+1);  
  
if (abs(mean_gz)<=giro_deadzone) ready++;  
else gz_offset=gz_offset-mean_gz/(giro_deadzone+1);  
  
if (ready==6) break;  
}  
}
```

Nilai “*offset*” yang didapatkan dari kode program tersebut digunakan untuk kode program sensor IMU pada *training kit flight controller* sehingga nilai sudut *pitch* dan *roll* pada posisi datar adalah nol. Nilai “*offset*” terdiri dari accelX, accelY, accelZ, gyroX, gyroY, dan gyroZ untuk digunakan pada program sensor IMU berikut.

```
mpu.setXGyroOffset(-5);  
mpu.setYGyroOffset(32);  
mpu.setZGyroOffset(-4);  
mpu.setXAccelOffset(1382);  
mpu.setYAccelOffset(1828);  
mpu.setZAccelOffset(702);
```

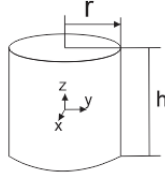
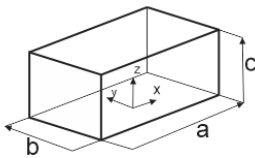
D. Pengujian *Training Kit Flight Controller*

Pengujian *training kit flight controller* dilakukan setelah wiring kabel dan *upload* program selesai. Pengoperasian dilakukan dengan menghidupkan remote kendali (*transmitter*) yang secara otomatis akan langsung terhubung dengan *receiver* apabila mengikuti langkah sebelumnya. Pengoperasian *training kit flight controller* dilakukan dengan melakukan pengujian pada fungsi kontrol setiap tuas pada remote kendali (*transmitter*) seperti pergerakan motor servo dan pergerakan motor BLDC. Pengujian sistem kendali dilakukan dengan langkah sebagai berikut.

1. Perhitungan Momen Inersia

Momen inersia adalah ukuran kelembaman suatu benda terhadap perubahan kedudukan atau posisi dalam gerak rotasi pada sumbunya. Setiap benda memiliki momen inersia yang dihasilkan dari rotasi pada sumbu x , y dan z . Persamaan momen inersia disesuaikan dengan bentuk komponen yang akan diukur. Terdapat dua bentuk dasar yang digunakan untuk menghitung inersia benda yaitu bentuk balok atau kubus dan silinder. Persamaan yang digunakan terdapat pada tabel 2 berikut.

Tabel 2. Persamaan Momen Inersia

Bentuk	Gambar	Persamaan
Silinder		$I_{G_{xx}} = I_{G_{yy}} = \frac{1}{12} m (3r^2 + h^2)$ $I_{G_{zz}} = \frac{1}{2} m r^2$
Balok atau Kubus		$I_{G_{xx}} = \frac{1}{12} m (b^2 + c^2)$ $I_{G_{yy}} = \frac{1}{12} m (a^2 + c^2)$ $I_{G_{zz}} = \frac{1}{12} m (a^2 + b^2)$

Inersia pesawat dapat dicari dengan membagi beberapa komponen dengan setiap bagian memiliki inersia terhadap sumbu x , y dan z . Pembagian komponen-komponen inersia *pesawat* lengkap beserta ukuran dimensi dan massa disajikan dalam tabel 3 berikut.

Tabel 3. Komponen Inersia *Training Kit* Pesawat *Flying-Wings*

No	Komponen	Panjang	Lebar	Tinggi	Massa	Jenis
1	Kotak Tengah	12 cm	20 cm	4 cm	477 gr	Balok
2	Sayap 1 kiri	38.5 cm	15 cm	2,5 cm	20 gr	Balok
3	Sayap 2 kiri	38.5 cm	23.5 cm	2,5 cm	60 gr	Balok
4	Sayap 1 kanan	38.5 cm	15 cm	2,5 cm	20 gr	Balok
5	Sayap 2 kanan	38.5 cm	23.5 cm	2,5 cm	60 gr	Balok

a. Perhitungan inersia sumbu x pesawat I_{xx}

Inersia sumbu x pesawat I_{xx} = inersia sumbu x kotak tengah (I_{KTx}) + inersia sumbu x sayap 1 kiri (I_{SKR1x}) + inersia sumbu x sayap 2 kiri (I_{SKR2x}) + inersia sumbu x sayap 1 kanan (I_{SKN1x}) + inersia sumbu x sayap 2 kanan (I_{SKN2x})

$$I_{xx} = (I_{KTx}) + (I_{SKR1x}) + (I_{SKR2x}) + (I_{SKN1x}) + (I_{SKN2x})$$

$$I_{xx} = 1,653 \times 10^{-3} + 3,854 \times 10^{-5} + 2,792 \times 10^{-4} + 3,854 \times 10^{-5} + 2,792 \times 10^{-4}$$

$$I_{xx} = 2,289 \times 10^{-3} \text{ kg.m}^2$$

b. Perhitungan inersia sumbu y pesawat I_{yy}

Inersia sumbu y pesawat I_{yy} = inersia sumbu y kotak tengah (I_{KTy}) + inersia sumbu y sayap 1 kiri (I_{SKR1y}) + inersia sumbu y sayap 2 kiri (I_{SKR2y}) + inersia sumbu y sayap 1 kanan (I_{SKN1y}) + inersia sumbu y sayap 2 kanan (I_{SKN2y})

$$I_{yy} = (I_{KTy}) + (I_{SKR1y}) + (I_{SKR2y}) + (I_{SKN1y}) + (I_{SKN2y})$$

$$I_{yy} = 6,360 \times 10^{-4} + 2,480 \times 10^{-4} + 7,442 \times 10^{-4} + 2,480 \times 10^{-4} + 7,442 \times 10^{-4}$$

$$I_{yy} = 2,620 \times 10^{-3} \text{ kg.m}^2$$

c. Perhitungan inersia sumbu z pesawat I_{zz}

Inersia sumbu z pesawat I_{zz} = inersia sumbu z kotak tengah (I_{KTz}) + inersia sumbu z sayap 1 kiri (I_{SKR1z}) + inersia sumbu z sayap 2 kiri (I_{SKR2z}) + inersia sumbu z sayap 1 kanan (I_{SKN1z}) + inersia sumbu z sayap 2 kanan (I_{SKN2z})

$$I_{zz} = (I_{KTz}) + (I_{SKR1z}) + (I_{SKR2z}) + (I_{SKN1z}) + (I_{SKN2z})$$

$$I_{zz} = 2,162 \times 10^{-3} + 2,845 \times 10^{-4} + 10,172 \times 10^{-4} + 2,845 \times 10^{-4} + 10,172 \times 10^{-4}$$

$$I_{zz} = 4,765 \times 10^{-3} \text{ kg.m}^2$$

2. Perhitungan Nilai *Full State Feedback Gain K*

Perhitungan nilai *full state feedback gain K* untuk setiap state yang terlibat dicari dengan penalaan nilai elemen **Q**. Variasi pembobotan elemen **Q** terlebih dahulu dilakukan perhitungan secara simulasi dengan menggunakan *Matlab* untuk memperoleh nilai gain **K**. Nilai gain **K** yang telah diperoleh selanjutnya dicobakan langsung pada program *hardware pesawat*. Dalam pembobotan nilai elemen **Q** membutuhkan nilai penyelesaian **P**. Nilai **P** dicari dengan menggunakan persamaan *Riccati*, sehingga untuk mendapatkan nilai **K** maka simulasi dijalankan dengan program *Matlab* seperti yang terlihat pada gambar berikut.

```
1 S = are(A,B*inv(R)*B',Q)
2 K = inv(R)*B'*S;
```

Gambar 12. Kode Program Perhitungan Nilai K di Matlab

Berdasarkan kode program yang tertulis pada gambar tersebut, terdapat sintak `are` dan `inv`. Sintak `are` merupakan fungsi yang telah disediakan oleh *Matlab* dalam menyelesaikan persamaan *Algebraic*

Riccati, sedangkan sintak `inv` merupakan fungsi yang disediakan oleh *Matlab* untuk menginvers suatu matriks.

3. Simulasi Matlab

Sudut *roll* dan *pitch* harus dikendalikan untuk menstabilkan gerak rotasi pesawat. Tahap pengujian dilakukan dengan simulasi pada aplikasi Matlab, untuk memperoleh respon kendali yang sesuai dengan kebutuhan minimum sistem. Respon sistem yang diperoleh berupa *rise time*, *settling time*, *overshoot*, *undershoot* dan *steady state error* yang digunakan sebagai dasar penentuan nilai **Q**. Penalaan nilai **Q** untuk setiap *state* rotasi dilakukan secara terpisah dan diubah baik ditambah atau dikurangkan untuk mendapatkan respon terbaik sistem. Nilai **Q** ini nantinya akan menghasilkan nilai *gain* **K**. Nilai *gain* **K** dimanfaatkan oleh sistem kendali dalam menstabilkan gerak terbang *pesawat* sesuai *x ref* atau *state* referensi yang diinginkan.

```

Editor - lqr_jst_flying_wing.m
lqr_jst_flying_wing.m
1 - Ixx = 0.002289183;
2 - Iyy = 0.002620667;
3 - Izz = 0.004765983;
4
5 - phi_a = (Iyy-Izz)/Ixx;
6 - theta_a = (Izz-Ixx)/Iyy;
7
8 - phi_b = 1/Ixx;
9 - theta_b = 1/Iyy;
10
11 - A = [0 1 0 0; 0 0 0 phi_a; 0 0 0 1; 0 theta_a 0 0];
12 - B = [0 0; phi_b 0; 0 0; 0 theta_b];
13 - C = [1 0 0 0; 0 0 1 0];
14 - D = 0;
15
16 - Q = [1 0 0 0; 0 0 0 0; 0 0 0 0; 0 0 0 0];
17 - R = 1;
18
19 - S = are(A,B*inv(R)*B',Q);
20 - K = inv(R)*B'*S;
21
22 - sys = ss(A-B*K, B, C, D);
23
24 - figure(1);
25 - step(sys);

```

Gambar 13. Kode Program Simulasi Matlab Kendali LQR

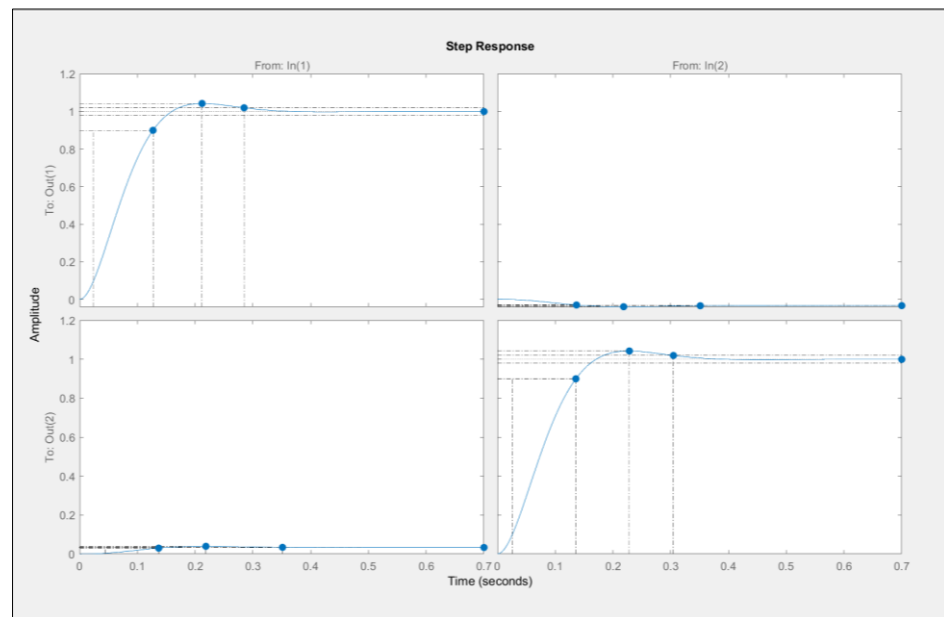
Sistem kendali LQR dalam mendapatkan *nilai full state feedback gain* **K** setiap *statenya*. Nilai terbaik **K** dipengaruhi oleh variasi

pembobotan elemen Q yang semakin besar maka akan menghasilkan nilai $gain$ K yang semakin besar juga. Semakin besar nilai $gain$ K akan menghasilkan torsi pada sistem *pesawat* yang juga semakin besar sehingga menjadikan sistem lebih responsif dibandingkan pemberian nilai $gain$ K kecil.

	1	2	3	4	5
1	0.9995	0.0677	0.0332	1.7674e-04	
2	-0.0332	1.5438e-04	0.9995	0.0724	
3					
4					
5					

Gambar 14. Hasil Simulasi Matlab untuk Nilai K

Hasil $gain$ K yang didapat dari pembobotan nilai elemen Q dibuktikan dengan data-data hasil simulasi yang diplotkan sebagai grafik. Data-data tersebut dianggap telah mewakili karakteristik respon sistem dengan ditunjukkan dari perolehan *rise time*, *settling time*, *overshoot*, *undershoot* dan *steady state error* yang terjadi dalam sistem *pesawat*.



Gambar 15. Grafik Karakteristik Respon Sistem Kendali LQR

Tabel 4. Karakteristik Respon Sistem Kendali LQR

Respon Transien	Hasil Simulasi Pengendalian <i>Roll</i>	
	Pengujian	Spesifikasi Kebutuhan
<i>Rise time (tr)</i>	0,103	< 1 detik
<i>Settling time (ts)</i>	0,285	< 3 detik
<i>Overshoot</i>	4,29	< 4,50
<i>Steady state error</i>	0,999	$\pm 4,5^0$
Respon Transien	Hasil Simulasi Pengendalian <i>Pitch</i>	
	Pengujian	Spesifikasi Kebutuhan
<i>Rise time (tr)</i>	0,11	< 1 detik
<i>Settling time (ts)</i>	0,305	< 3 detik
<i>Overshoot</i>	4,29	< 4,50
<i>Steady state error</i>	0,999	$\pm 4,5^0$

Berdasarkan tabel diatas, *state roll* dan *state pitch* pesawat memiliki karakteristik respon yang memenuhi spesifikasi minimum kestabilan sistem. Berdasarkan pengujian gerak rotasi *roll* dan *pitch*, pesawat telah memiliki karakteristik minimum kestabilan sistem, hal itu dibuktikan dengan data hasil pengujian yang telah dilakukan.

4. Implementasi Algoritma Kestabilan Gerak Rotasi

Mengendalikan gerak rotasi dapat diartikan mengatur torsi yang dihasilkan dari setiap motor servo yang mengatur sudut aileron dan elevator pada masing-masing sayap pesawat. Seperti yang telah dijelaskan dalam analisa sistem, torsi yang bekerja di dalam sistem pesawat meliputi torsi sumbu x disebut dengan *rolling*, dan torsi pada sumbu y atau *pitching*. Besar kedua torsi didapatkan melalui persamaan kendali LQR yang diwakili oleh u_2 untuk sinyal kendali *roll*, dan u_3 adalah sinyal kendali

pitch. Pengaplikasian kendali LQR pada hardware mikrokontroler diwujudkan dengan infrastruktur bahasa pemrograman C++ melalui potongan kode program seperti yang dipaparkan pada gambar 12 berikut.

```
1. u2 = ((roll_deg + roll_input*-1)*gain_roll) +  
    (gyroX_filt*gain_vel_roll);  
2. u3 = ((pitch_deg - (pitch_input*-1))*gain_pitch) +  
    (gyroY_filt*gain_vel_pitch);  
3. u2 = constrain(u2, -40,40);  
4. u3 = constrain(u3, -40,40);
```

Gambar 16. Potongan Kode Program Pengendalian Gerak Rotasi

Sinyal kendali u_2 dan u_3 dibentuk secara berurutan oleh hasil *sensor fusion* sudut *roll*, sudut *pitch* dan pembacaan kecepatan sudut oleh *gyroscope* untuk sumbu putar x dan y. Setiap *state* dikalikan dengan *gain* yang dipanggil dari matriks pada gambar 12. Variabel matriks baris satu kolom enam merupakan *gain* K untuk sinyal kendali u_2 *state* sudut *roll* dan kecepatan sudut *roll*. Kemudian variabel matriks baris dua kolom delapan merupakan *gain* K untuk sinyal kendali u_3 *state* sudut *pitch* dan kecepatan sudut *pitch*.

5. Pengujian Gerak Elevon (Mode Manual)

Pengujian gerak elevon dilakukan untuk memastikan sistem kendali manual dengan *transmitter* bekerja sesuai dengan karakteristik terbang pesawat. Pengujian dilakukan dengan menghidupkan sistem pada pesawat dan kemudian mengaktifkan CH5 pada remote kendali atau *transmitter*, yang berarti pesawat pada posisi *arming*.



Gambar 17. Posisi CH5 dan CH6 pada Remote Kendali

Kendali manual menggunakan *transmitter* memiliki karakteristik pergerakan elevon ketika pesawat naik, turun, belok kanan, dan belok kiri sebagai berikut.

Tabel 4. Gerakan Elevon (Mode Manual)

Posisi Tuas <i>Roll</i>	Posisi Elevon Kanan	Posisi Elevon Kiri
Kiri	Turun	Naik
Tengah	Tetap	Tetap
Kanan	Naik	Turun
Posisi Tuas <i>Pitch</i>	Posisi Elevon Kanan	Posisi Elevon Kiri
Tuas Pitch Posisi Bawah	Naik	Naik
Tuas Pitch Posisi Tengah	Tetap	Tetap
Tuas Pitch Posisi Atas	Turun	Turun

6. Pengujian Gerak Elevon (Mode *Stabilize*)

Pengujian gerak elevon pada mode *stabilize* dilakukan untuk memastikan sistem kendali dapat bekerja sesuai dengan algoritma yang dibuat. Mode *stabilize* adalah mode dimana pesawat akan mempertahankan kestabilan posisi *roll* dan *pitch* dengan menggunakan elevon. Mode *stabilize* hanya memanfaatkan data dari sensor IMU (*accelerometer* dan *gyroscope*) dengan metode DMP (*Digital Motion Processing*). Pengujian dilakukan dengan menghidupkan sistem pada pesawat dan kemudian mengaktifkan CH6 pada remote kendali atau *transmitter*, yang berarti pesawat pada posisi *stabilize mode*. Pergerakan elevon pada mode *stabilize* akan berlawanan dengan gerakan pesawat, hal tersebut dilakukan untuk mengembalikan pesawat pada posisi yang lurus. Gerakan elevon pada mode *stabilize* ditampilkan pada tabel berikut.

Tabel 5. Gerakan Elevon (Mode *Stabilize*)

Posisi Pesawat	Posisi Elevon Kanan	Posisi Elevon Kiri
Tengah	Tetap	Tetap
Miring Kiri	Naik	Turun
Miring Kanan	Turun	Naik
Miring Bawah	Naik	Naik
Miring Atas	Turun	Turun

E. Aplikasi Robot *Flying Wings* UAV

Robot *flying wings* terus berkembang seiring dengan kemajuan teknologi dan inovasi di bidang *aerial robotic* atau *Unmanned Aerial Vehicle*. Robot *flying wings* memiliki beragam aplikasi di berbagai bidang sebagai berikut.

1. Pemantauan dan Pengawasan

Robot *flying wings* dapat digunakan untuk misi pemantauan dan pengawasan dalam berbagai lingkungan, seperti pemantauan kebakaran hutan, pemantauan keadaan cuaca, pemantauan perbatasan wilayah, dan pengawasan infrastruktur hingga lalu lintas.

2. Pemetaan dan Survei

Robot *flying wings* dapat digunakan untuk industri pemetaan seperti survei lahan, hutan, pertanian, serta pemetaan daerah yang sulit diakses dengan cepat dan efisien.

3. Penyediaan Layanan Darurat

Robot *flying wings* dapat digunakan untuk memberikan layanan darurat dalam situasi bencana atau keadaan darurat, seperti pencarian dan penyelamatan, serta pengiriman bantuan medis dan logistik di area bencana yang sulit dijangkau oleh kendaraan umum.

4. Pertanian dan Konservasi

Robot *flying wings* dapat digunakan dalam pertanian dengan area yang luas, seperti memantau tanaman, mengidentifikasi hama, atau mendukung praktik pertanian dengan lebih efektif dan efisien.

5. Pengawasan Lingkungan

Robot *flying wings* dapat digunakan untuk mengumpulkan data lingkungan, seperti pemantauan kualitas udara, pencemaran lingkungan, dan pemantauan keanekaragaman hayati.

PENUTUP

Buku panduan ini dirancang dengan tujuan untuk memberikan pemahaman yang jelas dan praktis dalam pemasangan, penggunaan, dan pengoperasian *training kit flight controller*. Peneliti berharap dengan adanya buku panduan ini dapat memberikan banyak manfaat dan kontribusi positif bagi para pengguna, terutama dalam pengembangan *training kit flight controller robot flying wings* UAV sebagai media pembelajaran mata kuliah robotika. Peneliti menyadari bahwa dalam dunia teknologi dan robotika, hal-hal berkembang dengan sangat cepat, sehingga peneliti berharap dengan adanya buku panduan ini dapat memberikan semangat bagi para pengguna untuk terus melakukan pembaharuan di bidang *aerial robotic* dengan informasi yang relevan dan terbaru.

Terima kasih kepada seluruh pihak yang telah berkontribusi dalam penyusunan buku panduan ini. Semoga buku ini dapat menjadi sumber pengetahuan yang bermanfaat dan dapat menginspirasi para pengguna untuk mengembangkan inovasi di bidang *aerial robotic*. Selamat berkembang dan semoga sukses!

Yogyakarta, 25 Juli 2023
Penulis,

Zulhakim Seftiyana Roviyan
NIM. 19502241009

DAFTAR PUSTAKA

- Ardiantara, P., Sumiharto, R., & Wibowo, S. B. (2014). Purwarupa Kontrol Kestabilan Posisi dan Sikap pada Pesawat Tanpa Awak Menggunakan IMU dan Algoritma Fusion Sensor Kalman Filter. *IJEIS*, 4(1), 25–34.
- Salim, A. I., Saragih, Y., & Hidayat, R. (2020). IMPLEMENTASI MOTOR SERVO SG 90 SEBAGAI PENGGERAK MEKANIK PADA E. I. HELPER (ELECTRONICS INTEGRATION HELMET WIPER). *Jurnal Electro Luceat*, 6(2).
- Tafonao, T. (2018). PERANAN MEDIA PEMBELAJARAN DALAM MENINGKATKAN MINAT BELAJAR MAHASISWA. *Jurnal Komunikasi Pendidikan*, 2(2), 103–114.
- Yasa, Y. (2022). An Efficient Brushless DC Motor Design for Unmanned Aerial Vehicles. *European Journal of Science and Technology*, 288–294.